Journal of Process Control 161 (2026) 103679



Contents lists available at ScienceDirect

Journal of Process Control

journal homepage: www.elsevier.com/locate/jprocont





Optimising kernel-based multivariate statistical process control

Zina-Sabrina Duma *, Victoria Jorry, Tuomas Sihvonen, Satu-Pia Reinikainen, Lassi Roininen

LUT University, Yliopistonkatu 34, FI-53850, Lappeenranta, Finland

ARTICLE INFO

Toolbox link: https://github.com/sab-in-science/k-mspc

Keywords:
MSPC
Kernel PCA
Kernel PCR
Kernel flows

ABSTRACT

Kernel-based multivariate statistical process control (K-MSPC) extends classical MSPC by capturing nonlinear dependence through kernel principal component analysis (K-PCA). In practice, however, K-MSPC performance depends strongly on the choice of kernel and hyperparameters, which are often selected using non-procedural and time-consuming searches (e.g., grid/line search) that do not scale to richer parameterisations. We propose a fully procedural kernel-learning framework for K-MSPC inspired by Kernel Flows, in which kernel parameters are learned using stochastic subsampling and gradient-based optimisation. Because MSPC relies on a truncated K-PCA subspace, we optimise kernel parameters through an intermediate kernel principal component regression (K-PCR) discrimination objective that is structurally aligned with the latent-variable representation used by monitoring statistics. Building on this framework, we demonstrate extensions that become computationally feasible: learning variable-wise (additive) kernel parameters and learning kernel combinations to adapt the kernel family to data. The approach is evaluated on the Tennessee Eastman Process benchmark. Results show improved fault detection, including difficult faults that are poorly detected by standard K-PCA baselines, while providing diagnostic insights via variable contribution and attribution analyses. The proposed optimisation requires access to faulty (or simulated faulty) data and thus represents a supervised/semi-supervised calibration strategy complementary to classical unsupervised MSPC.

1. Introduction

Multivariate Statistical Process Control (MSPC) is a statistical framework that simultaneously monitors and controls processes with interrelated variables [1]. Unlike univariate methods that focus on individual parameters, MSPC considers variable correlations, providing a comprehensive view of the behaviour of the process [2]. This allows the detection of subtle shifts and anomalies that might be missed when examining variables in isolation [3].

MSPC is effectively used across industries, particularly in the chemical sector [4], where it concurrently analyses multiple process parameters to ensure product quality and efficiency [5]. In semiconductor manufacturing, MSPC manages fabrication processes in which various variables affect the performance of the final product [6]. In addition, MSPC is applied in pharmaceutical production to ensure consistent drug quality by monitoring critical quality attributes during manufacturing [7].

While MSPC offers benefits, it has limitations. Traditional techniques often assume linear relationships and normal distribution, which may not apply in all situations [8]. MSPC can also struggle to detect small shifts in process mean or variance, especially with high-dimensional data and complex nonlinear interactions [9]. These issues highlight the need for advanced methodologies like Kernel MSPC to enhance its effectiveness in diverse industrial contexts [10].

Kernel MSPC extends traditional MSPC techniques by addressing the limitations of linear assumptions [11]. It uses kernel functions to transform process data into a higher-dimensional feature space, allowing for the detection of complex, nonlinear relationships and subtle variations that linear methods may overlook [12]. Kernel MSPC also has its limitations. Kernel MSPC relies on the choice of a kernel function and its associated hyperparameters, and many valid kernel constructions exist (including composite kernels with multiple hyperparameters). In practice, however, kernel-MSPC studies often restrict attention to a small set of standard kernels (most commonly Gaussian/RBF, occasionally polynomial or Matérn) and tune one or a few global hyperparameters using non-procedural approaches such as manual tuning, line search, or coarse grid search. This makes kernel calibration time-consuming and difficult to reproduce, and it rarely supports richer parameterisations such as variable-wise kernel parameters or kernel combinations learned from data. The computational complexity in high-dimensional feature spaces can also limit real-time applications with large datasets. Additionally, interpreting results can be challenging due to transformations into higher-dimensional spaces, complicating the identification of variables linked to detected anomalies [13].

* Corresponding author.
E-mail address: Zina-Sabrina.Duma@lut.fi (Z.-S. Duma).

https://doi.org/10.1016/j.jprocont.2026.103679
Received 13 November 2025; Received in revised form 16 February 2026; Accepted 21 February 2026
Available online 3 March 2026
0959-1524/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Beyond traditional MSPC and kernel MSPC approaches, there has been substantial recent interest in machine learning (ML)-based methods for process monitoring and control charts that go beyond classical statistical assumptions. For example, new machine-learning control chart frameworks that combine neural networks, random forests, and support vector machines have been proposed for simultaneous monitoring of multivariate process parameters, demonstrating improved detection and identification performance compared to classical charts [14]. Similarly, ML-informed frameworks have been developed for monitoring structured multistage processes using state-space and data-driven models [15]. Deep learning has also been applied to control chart design for autocorrelated processes, such as LSTM-based monitoring that captures complex temporal dependencies in multivariate process data [16]. Broad reviews of AI and ML algorithms in statistical process monitoring highlight the proliferation of neural networks (ANN, CNN, RNN, GAN) and related models for various SPC tasks, from classification and pattern recognition to anomaly detection in high-dimensional and nonstationary data streams [17]. Furthermore, self-supervised learning approaches for MSPC have been introduced, leveraging representation learning to detect and localise process anomalies in discrete manufacturing contexts [18].

While these approaches primarily aim to learn flexible decision functions or latent representations optimised directly for fault classification or anomaly scoring, they often depart from the classical MSPC paradigm based on control charts, monitoring statistics, and variance decomposition. In addition, many ML-based SPC methods rely on high-capacity models and large labelled datasets, and provide limited interpretability in terms of variable-wise contributions to monitoring statistics. In contrast, the present work does not propose a new monitoring model, nor does it aim to replace control-chart-based SPC with a classifier. Instead, it focuses on the procedural learning of kernel parameters for kernel MSPC, such that the resulting kernel PCA representation improves T^2 and SPE_x monitoring performance while retaining statistical interpretability.

Optimising kernel parameters in Kernel-based MSPC is essential for effective process monitoring and fault detection. Traditional methods use grid search or cross-validation to evaluate combinations of kernel parameters, aiming to find settings that minimise monitoring errors [19]. Advanced optimisation algorithms like Bayesian optimisation and genetic algorithms efficiently explore parameter spaces, utilising an optimisation function that minimises the false alarms and maximises the correct identification of faulty observations [20]. Kernel MSPC is not restricted to any single kernel family; in principle, practitioners may employ a wide range of stationary/non-stationary kernels, composite kernels, and kernels with multiple hyperparameters. In much of the kernel-MSPC literature, three practical limitations remain: (i) search-based tuning can be time-inefficient and irreproducible; (ii) kernel function selection is typically fixed a priori rather than learned; and (iii) hyperparameters are usually shared globally across variables, which can be suboptimal for heterogeneous process measurements and difficult faults. Given these limitations, there is a need for a more procedural kernel learning procedure for Kernel MSPC.

Kernel Flows is a kernel learning framework originally introduced to adapt kernel parameters based on stability properties of kernel interpolants under data subsampling [21]. Rather than introducing a new optimisation algorithm, Kernel Flows defines a principled kernel-learning objective, which is typically minimised using stochastic gradient descent. By iteratively updating kernel parameters to improve robustness with respect to subsampling, the method promotes kernel representations that generalise well. Kernel Flows has been successfully applied in regression and classification settings, demonstrating improved accuracy and robustness in kernel-based latent variable models [22,23].

In this work, we adopt a Kernel-Flows-inspired optimisation framework to learn kernel parameters for kernel-based MSPC. Since the original Kernel Flows objective is formulated for full-rank kernel interpolants in Gaussian Process Regression, it is not directly compatible with kernel MSPC, which relies on a truncated kernel PCA representation. To ensure consistency between the optimisation objective and the monitoring task, we introduce an intermediate classification step that discriminates between healthy and faulty process states, with Kernel Principal Component Regression (K-PCR). Kernel parameters are optimised using either (a) subsampling-based, classification-driven loss, or (b) composite MSPC fault detection & false alarm rate, while stochastic gradient descent is employed solely as the numerical optimisation tool.

The primary contribution of this work is therefore not a new optimisation algorithm, but a fully procedural and scalable kernel-learning framework for kernel MSPC that is structurally aligned with MSPC monitoring objectives. This directly addresses a key limitation of existing kernel MSPC methods, where kernel parameters are typically selected using non-procedural and computationally intensive techniques such as grid or line search.

Building on this optimisation framework, we demonstrate several secondary contributions that become computationally feasible as a result: (i) learning individual kernel parameters for each process variable, enabling improved detection of difficult faults; (ii) learning kernel combinations to automatically adapt the kernel function to the data; and (iii) leveraging the optimised model parameters for improved interpretability via variable contribution and attribution analyses. It should be explicitly noted that the proposed kernel optimisation framework requires faulty data for calibration and therefore departs from the fully unsupervised paradigm of traditional MSPC.

The method is illustrated with various case studies of simulated faults from the benchmark fault detection Tennessee Eastman Process (TEP) dataset [24]. In the widely used TEP benchmark, many studies report that linear PC-based monitoring fails to capture nonlinear dependencies and multimodal data structures. For example, linear PCA was shown to underperform when compared to auto-encoders or kernel-PCA approaches on TEP, because the non-linear relationships are not captured [25,26]. Similarly, the PCA relies on underlying Gaussian-score assumptions and struggles in the dynamic, multimodal environment of TPE [27].

The article is structured as follows: Section 2 presents the process data and the base mathematical methods utilised, Section 3 presents the results of the KF-optimised MSPC, and Section 4 concludes the findings.

2. Materials and methods

2.1. Process and data description

In this study, the TEP is utilised, with the faults simulated as in Russell et al. (2000) [24]. On this well-established benchmark process, the proposed methods will be applied to demonstrate their efficiency and to aid in further method comparisons. The TE process model is a realistic simulation program of a chemical plant widely accepted as a benchmark for control and monitoring studies. The process is described in detail in [28], and has five significant units: reactor, condenser, compressor, separator, and stripper. The process has two products from 4 reactants, an inert and a by-product, making a total of 8 components denoted **A**-**H**. The process measures 52 variables, which are expanded in Appendix A, of which 41 are process variables and 11 are manipulated variables.

From the total 21 faults presented in [24], four have been selected for main visualisation and in-depth analysis in the present paper (summarised in Table 1), with a summary of the fault detection rates presented for all the faults.

Because prior information on the mathematical model of the TE process is not available, the Process Monitoring and Fault Diagnosis (PM-FD) system must be constructed solely based on process data. The data sets in [29] are commonly accepted for PM-FD studies.

Table 1
Descriptions of considered faults in the TE process. * Faults that were not detected in the original study through conventional methods. [24].

Fault number	Fault ID	Variable	Fault affected
1	E1	A/C feed ratio, B composition constant	step
2	E2	B composition, A/C ratio constant	step
3	D1*	D feed temperature	step
9	D2*	D feed temperature	random variation

The data are divided into three disjoint partitions. For each of the partitions, the process variables are simulated every three minutes. The first consists of normal-operation data, in which the process operates under healthy conditions throughout. The second consists of faulty-operation calibration datasets, where each dataset contains samples generated entirely under a specific simulated fault condition. These two partitions are used exclusively for kernel parameter optimisation and model calibration.

A third, independent test partition is used for evaluation. In these datasets, the process starts under normal operation and a fault is introduced after 160 (eight simulation hours) observations, simulating an online monitoring scenario. These test datasets are not used during kernel parameter optimisation or model calibration and serve solely for performance evaluation and benchmarking.

2.2. Mathematical methods

This section presents the mathematical methods used to generate the MSPC control charts, both in the traditional and kernel PCA versions. Also, the mathematical principles for the parameter optimisation are detailed.

2.2.1. Principal component analysis (PCA) and kernel PCA

PCA [30] decomposes a data matrix as

$$\mathbf{X} = \mathbf{T}\mathbf{P}^{\intercal} + \mathbf{E}, \tag{1}$$

where $\mathbf{T}$ contains the latent scores, $\mathbf{P}$ the loadings, and $\mathbf{E}$ the residual variation.

K-PCA [20,31] extends PCA by performing the decomposition in a reproducing kernel Hilbert space (RKHS) $\mathscr{H}$ via an implicit nonlinear feature map $\phi(\cdot)$. The inner products in $\mathscr{H}$ are computed using a kernel function

$$\mathsf{k}(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle, \tag{2}$$

allowing nonlinear structure in the original data space to be captured without explicit feature construction. For completeness, the mathematical derivation of K-PCA is provided in Appendix B.

To map the data into $\mathscr{H}$, different kernels can be employed. A choice the Matérn kernel,

$$\mathbf{k}_M(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}) = \gamma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu} \frac{\|\mathbf{x} - \mathbf{y}\|}{\sigma} \right)^{\nu} K_{\nu} \left(\sqrt{2\nu} \frac{\|\mathbf{x} - \mathbf{y}\|}{\sigma} \right), \tag{3}$$

where $\boldsymbol{\theta} = \{\gamma^2, \sigma, \nu\}$. As $\nu \to \infty$, the Matérn kernel converges to the Gaussian kernel

$$\mathsf{k}_G(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}) = \gamma^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2} \right), \tag{4}$$

while heavier-tailed behaviour can be obtained using, for example, the Cauchy kernel

$$\mathsf{k}_C(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}) = \gamma^2 \left(1 + \frac{\|\mathbf{x} - \mathbf{y}\|^2}{\sigma^2} \right)^{-1}. \tag{5}$$

Table 2
Interpretation of variable-wise kernel parameters in the additive kernel formulation. The amplitude parameter γ_l^2 controls the weight of each variable in the kernel representation, while scale and smoothness parameters determine how variability in that variable is expressed.

Kernel	Amplitude γ_l^2	Scale σ_l	Smoothness ν
Gaussian (RBF)	↑: increases contribution of variable l to kernel similarity ↓: reduces influence of variable	↑: smoother similarity; only large-scale deviations affect the model ↓: increased sensitivity to small, local deviations	–
Cauchy	↑: increases variable contribution with heavy-tailed influence ↓: reduces variable impact	↑: slower decay of similarity; large deviations remain influential over a wider range ↓: faster decay; emphasis on small, local deviations	– –
Matérn	↑: increases contribution of variable l to kernel similarity ↓: reduces influence of variable	↑: larger effective scale; smoother similarity ↓: increased sensitivity to fine-scale variation	↑: smooth, trend-dominated behaviour ↓: rough behaviour; sensitivity to abrupt changes

When the optimal kernel structure is not known a priori, kernels can be combined in an additive form [23]. Additive kernel constructions of this form are closely related to kernelised Generalised Additive Models (GAMs), where the response is modelled as a sum of feature-wise nonlinear functions. Such formulations have been studied in the machine learning literature for improved interpretability and flexibility [32,33]. In this work, we employ additive kernels both with variable-wise parameters,

$$\mathsf{k}(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}) = \sum_{l=1}^{d} k(x_l, y_l; \boldsymbol{\theta}_l), \tag{6}$$

where d denotes the number of process variables and $\boldsymbol{\theta}_l$ controls the kernel behaviour for variable l, and with a family of kernel types

$$\begin{aligned} \mathsf{k}(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}) = {} & \mathsf{k}_G(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}_G) + \mathsf{k}_{M1/2}(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}_{M1/2}) \\ & + \mathsf{k}_{M3/2}(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}_{M3/2}) + \mathsf{k}_{M5/2}(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}_{M5/2}) + \mathsf{k}_C(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta}_C), \end{aligned} \tag{7}$$

where k_G is computed as in Eq. (4), k_C is computed as in Eq. (5), and $\mathsf{k}_{M1/2}$, $\mathsf{k}_{M3/2}$ and $\mathsf{k}_{M5/2}$ are computations of Eq. (3) where ν takes the values $\frac{1}{2}$, $\frac{3}{2}$ and respectively $\frac{5}{2}$.

This formulation allows different variables to contribute with different nonlinear sensitivities, which is rarely exploited in kernel MSPC, as seen in Table 2.

The kernel matrix is centred in $\mathscr{H}$ using

$$\mathbf{K}_c = \left(\mathbf{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^{\intercal} \right) \mathbf{K} \left(\mathbf{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^{\intercal} \right), \tag{8}$$

where n is the number of samples.

Using additive kernels requires computing d one-dimensional kernel contributions per sample pair. For a dataset of n samples, the computational cost scales as $\mathcal{O}(dn^2)$, while the memory requirement remains $\mathcal{O}(n^2)$ since variable-wise contributions can be accumulated without storing separate kernel matrices.

2.2.2. Multivariate Statistical Process Control (MSPC) and kernel MSPC

Hotelling's T^2 and SPE_x control charts are used to monitor modelled and residual variation, respectively in traditional MSPC. For completeness, their mathematical definitions and control limit derivations are provided in Appendix C.

In the kernel-based MSPC framework, it is also possible to quantify the influence of each process variable on the overall Hotelling's T^2 statistic. Let $\mathbf{K}_c$ denote the centred kernel matrix, whose eigenvalue decomposition yields eigenvalues λ_h and eigenvectors $\mathbf{u}_h$ for $h = 1, \ldots, H$, corresponding to the retained principal components in the feature space. For a new observation $\mathbf{x}$, the centred kernel vector $\mathsf{k}_c(\mathbf{x}, \mathbf{X})$ is computed with respect to the training data $\mathbf{X}$, and the normalised kernel principal component scores are obtained as

$$y_h(\mathbf{x}) = \frac{\mathsf{k}_c(\mathbf{x}, \mathbf{X}; \boldsymbol{\theta})\, \mathbf{u}_h}{\sqrt{\lambda_h}}. \tag{9}$$

The overall Hotelling's T^2 index is then given by

$$T^2(\mathbf{x}) = \sum_{h=1}^{H} y_h(\mathbf{x})^2 = \sum_{h=1}^{H} \frac{\left(\mathsf{k}_c(\mathbf{x}, \mathbf{X}; \boldsymbol{\theta})\, \mathbf{u}_h\right)^2}{\lambda_h}. \tag{10}$$

When an additive kernel function is employed, as expressed in Eq. (6), the total centred kernel can be decomposed into variable-wise components $\mathsf{k}_c^{(l)}(\mathbf{x}, \mathbf{x}_l'; \theta_l)$ for each process variable $l = 1, \ldots, d$. The partial score of variable l on component h can then be defined as

$$y_h^{(l)}(\mathbf{x}) = \frac{\mathsf{k}_c^{(l)}(\mathbf{x}, \mathbf{x}_l'; \theta_l)\, \mathbf{u}_h}{\sqrt{\lambda_h}}, \tag{11}$$

and the variable contribution to the T^2 statistic as

$$r_l(\mathbf{x}) = \sum_{h=1}^{H} \left(y_h^{(l)}(\mathbf{x})\right)^2 = \sum_{h=1}^{H} \frac{\left(\mathsf{k}_c^{(l)}(\mathbf{x}, \mathbf{x}_l'; \theta_l)\, \mathbf{u}_h\right)^2}{\lambda_h}. \tag{12}$$

The normalised contribution coefficients, computed as $c_l(\mathbf{x}) = r_l(\mathbf{x}) / \sum_{m=1}^{d} r_m(\mathbf{x})$, represent the proportion of the total T^2 variation attributed to each process variable. These contributions provide a diagnostic attribution within the model of variable relevance in the detection of process faults, helping to identify the parameters that most strongly influence deviations from normal operation.

2.2.3. Kernel parameter optimisation

Kernel Flows is a kernel parameter learning technique proposed by Owhadi & Yoo (2019) [21], which utilises stochastic gradient descent and repeated random subsampling to learn kernel parameters that are stable under data reduction. In the original formulation, the optimisation objective is derived from the difference between a model calibrated using all data points and a model calibrated using a randomly selected subset of the data. In the original Kernel Flows formulation, the optimisation objective is derived from the reproducing kernel hilbert space (RKHS) norm of the kernel interpolant, which is required to remain stable under random subsampling of the training data.

The proposed method has similarities with the Kernel Flows approach, including its iterative and cross-validated learning, and parameter updating with stochastic gradient descent, and to the modified version in Lamminpää et al. 2024 [34] that has a Leave-One-Out (LOO) approach and computes an ℓ^2-norm loss. As in Lamminpää et al. (2024), the present work adopts the subsampling-based stability motivation of Kernel Flows while replacing the original RKHS-norm objective with a task-specific loss tailored to the target application. The method is summarised in Algorithm 1.

While Kernel Flows was originally introduced for hyperparameter learning in Gaussian Process Regression (GPR), directly applying a GPR-based optimisation objective is not well aligned with the structure of kernel MSPC. In GPR, all eigenmodes of the kernel matrix contribute to the regression or classification function, whereas in kernel MSPC only a truncated set of principal components is retained to construct the T^2 and SPE_x control charts.

As a result, kernel parameters that yield optimal classification performance in a full-rank GPR model do not necessarily produce a kernel PCA model whose retained components effectively separate normal and faulty operation. To ensure consistency between the optimisation objective and the monitoring task, we therefore employ K-PCR as an intermediate surrogate.

K-PCR is used as a regression-based discrimination surrogate, where kernel PCA scores are regressed onto binary dummy variables indicating healthy and faulty operation. This follows standard PCR-DA and PLS-DA formulations in chemometrics and is used here solely to guide kernel parameter learning rather than as a final classifier. Since both K-PCR discrimination and kernel MSPC monitoring are constructed from the same truncated kernel PCA subspace, optimising kernel parameters via K-PCR directly reshapes the latent directions that determine the T^2 and SPE_x statistics.

By performing classification in the kernel principal component space, K-PCR explicitly restricts the optimisation to the same low-dimensional subspace used for MSPC. Kernel parameters are thus learned to maximise discrimination between healthy and faulty states using the dominant systematic variation of the process, ensuring that the resulting kernel is well suited for constructing effective MSPC control charts.

Compared to the original Kernel Flows approach, the proposed method aims to produce process control limits by solving a discrimination problem that differentiates between normal and faulty operating states. Two loss functions are tested with the approach. The first loss function (ρ_a) for a sub-iteration s is

$$\rho_{a,s} = 1 - \frac{\eta_n^a + \eta_f^a}{2} \tag{13}$$

where η_n^a is the proportion of normal-functioning samples that are classified as normal ("class 0") and η_f^a is the proportion of faulty operation samples that are classified as faulty ("class 1"). Thus, the kernel parameters are optimised in a way that facilitates a good separation between the normal and faulty states of the process.

The second loss function is tested based on the correct monitoring rate (CMR) [20] instead of correct K-PCR classification, where

$$\rho_b = 1 - \frac{\eta_n^b + \eta_f^b}{2} \tag{14}$$

η_n^b is the proportion of normal-functioning samples that are under the alarm limit in MSPC, η_f^b is the proportion of faulty operation samples that are above the alarm limit in MSPC. The main difference between the cost functions in the optimiser Eq. (13) and the final evaluation Eq. (14) is that, in the optimiser, the classification rate is monitored, and in the latter case, the correct monitoring rate in the MSPC chart is followed.

Although both ρ_a and ρ_b involve normal and faulty samples, they optimise fundamentally different objectives. The loss ρ_a evaluates discrimination performance of a K-PCR classifier in the kernel principal component space, without reference to MSPC control limits. In contrast, ρ_b evaluates fault detection performance in the monitoring space by measuring the proportion of samples that fall below (normal operation) or exceed (faulty operation) the calibrated MSPC alarm limits. As a result, correct classification and correct monitoring are not equivalent, and optimising one does not necessarily guarantee optimal performance of the other.

The per-iteration cost of the optimisation is dominated by kernel construction and K-PCA. With S sub-iterations, batch size n, d process variables, and H retained principal components, the computational cost per iteration is approximately

$$\mathcal{O}\left(S\left(dn^2 + n^2 H\right)\right)$$

when using partial eigensolvers that retain only the leading components, or

$$\mathcal{O}\left(S\left(dn^2 + n^3\right)\right)$$

when dense eigendecomposition is employed. Hence, introducing variable-wise kernel parameters increases the kernel construction cost linearly with d, while the overall scaling with respect to the batch

Algorithm 1 Kernel parameter optimisation workflow

Input: Initial kernel parameters (θ_0), normal-functioning process data ($\mathbf{X}^{normal}$), faulty-functioning process data ($\mathbf{X}^{faulty}$), number of PCs (H), number of iterations (I), number of samples extracted in a sub-iteration (ns), number of sub-iterations (S), learning rate (α) ***Output:*** optimised kernel parameters (θ_I).

for $i = 1$ to I **do**
  for $s = 1$ to S **do**
    $\mathbf{X}_s^{normal} \leftarrow \mathbf{X}_{\pi,\cdot}^{normal}$ where π is an ns sample extraction from a random permutation.
    $\mathbf{X}_s^{faulty} \leftarrow \mathbf{X}_{\pi,\cdot}^{faulty}$
    $\mathbf{X}_s \leftarrow \left[\mathbf{X}_s^{faulty}; \mathbf{X}_s^{normal}\right]$
    $\mathbf{K} \leftarrow k(\mathbf{X}_s, \mathbf{X}_s; \boldsymbol{\theta}_i)$
    $\widetilde{\mathbf{K}} \leftarrow (\mathbf{I} - \frac{1}{N}\mathbf{1}_N\mathbf{1}_N^\intercal)\mathbf{K}(\mathbf{I} - \frac{1}{N}\mathbf{1}_N\mathbf{1}_N^\intercal)$
    $\mathbf{T} \leftarrow \mathrm{kpca}(\widetilde{\mathbf{K}}, H)$
    $\mathbf{y} \leftarrow \left[\mathbf{1}_{ns}; \mathbf{0}_{ns}\right]$
    $\mathbf{b} \leftarrow \mathbf{T}^\dagger\mathbf{y}$
    $\rho_s \leftarrow 1 - \frac{\eta_n+\eta_f}{2}$
  end for
  $\bar{\rho}_i \leftarrow \sum_{s=1}^{S} \rho_s$
  $\nabla_{\theta,i} \leftarrow \mathrm{diff}(\theta_i, \bar{\rho}_i)$
  $\theta_{i+1} \leftarrow \alpha\nabla_{\boldsymbol{\theta},i} f(\theta_i)$
end for

size n is governed by the K-PCA step. In practice, the computational burden is controlled by limiting n through subsampling and retaining a small number of principal components, which is compatible with online process monitoring requirements.

2.2.4. Variable contributions to the model

Because we are optimising a K-PCR problem (ρ_a), we can utilise, in addition to the variable contributions, an exact additive variable attribution derived from the kernel representation. Assume an additive kernel across variables. Let $\mathbf{X}_{\mathrm{tr}}$ be the training matrix used to fit K-PCR, $\mathbf{B} \in \mathbb{R}^{n_{\mathrm{tr}}}$ the kernel-PCR weights (obtained by KPCA on $\mathbf{K}_c$ followed by PCR), and define the centred cross-kernel row for a test sample $\mathbf{x}$ as

$$\mathbf{k}_c(\mathbf{x}, \mathbf{X}_{\mathrm{tr}}) \;=\; \sum_{l=1}^{p} \mathbf{k}_c^{(l)}(\mathbf{x}, \mathbf{X}_{\mathrm{tr}}), \qquad \mathbf{k}_c^{(l)}(\mathbf{x}, \mathbf{X}_{\mathrm{tr}}) \in \mathbb{R}^{1\times n_{\mathrm{tr}}}.$$

The K-PCR prediction is linear in the centred cross-kernel:

$$\hat{y}(\mathbf{x}) \;=\; \mathbf{k}_c(\mathbf{x}, \mathbf{X}_{\mathrm{tr}})\,\mathbf{B} \;=\; \sum_{l=1}^{p} \mathbf{k}_c^{(l)}(\mathbf{x}, \mathbf{X}_{\mathrm{tr}})\,\mathbf{B}.$$

Hence, the per-variable exact additive attribution for variable l is

$$\phi_l(\mathbf{x}) \;=\; \mathbf{k}_c^{(l)}(\mathbf{x}, \mathbf{X}_{\mathrm{tr}})\,\mathbf{B}, \qquad \text{so that} \qquad \sum_{l=1}^{p} \phi_l(\mathbf{x}) \;=\; \hat{y}(\mathbf{x}). \tag{15}$$

Eq. (15) provides an exact additive decomposition of the K-PCR prediction when the kernel is additive across variables and the predictor is linear in the centred cross-kernel. For visualisation, we report signed $\phi_l(\mathbf{x})$ or magnitudes $|\phi_l(\mathbf{x})|$. The proposed attribution should not be interpreted as a post-hoc explanation method. Instead, it follows directly from the linearity of K-PCR in the centred cross-kernel and the additive structure of the kernel across variables. As a result, the prediction is exactly decomposed into variable-wise terms without approximation. The attribution therefore quantifies how each process variable contributes to the discrimination between normal and faulty operation under the learned kernel representation.

A baseline "healthy mean" profile can be defined over the healthy partition $\mathcal{H}$ as

$$\bar{\phi}_l^{(\mathrm{H})} \;=\; \frac{1}{|\mathcal{H}|} \sum_{i\in\mathcal{H}} \left|\phi_l(\mathbf{x}_i)\right|. \tag{16}$$

Although supervised labels are used during kernel parameter optimisation, we emphasise that the procedure does not perform a full formal cross-validation of kernel hyperparameters. The truncated K-PCA representation (fixed number of retained components) and stochastic subsampling act as structural regularisers and provide a form of stability pressure during optimisation, but they should not be interpreted as a substitute for nested cross-validation when such validation is feasible. To limit optimistic bias, all reported monitoring results are evaluated on an independent test partition that is not used during optimisation or model calibration.

2.3. Proposed procedure

The diagram in Fig. 1 summarises the proposed procedure based on the mathematical methods presented. The process data is collected, both in normal-functioning and sub-optimal or faulty operations. Simulations of the process operation in possible faulty conditions can be utilised.

After there exists data on both the normal and faulty process, the kernel parameters are learned with the Kernel Flows-inspired procedure presented in the current study. The control charts are created with the K-PCA model with optimised parameters.

When new data arrives, it is projected onto the K-PCA model and further into the control charts. The position of the projections in relation to the control chart limits are evaluated, and the state of the process is determined (in-control or faulty). Further investigation on the causes of the fault can be conducted through contribution analysis, but the issue is not the subject of the present paper.

In our examples, the dataset has three parts: (a) a healthy-functioning monitoring, (b) a faulty-functioning calibration set and (c) a test partition, where the process starts normal-functioning and the fault develops after eight hours. The first two parts (a, b) will be utilised for model calibration and kernel parameter optimisation (Steps I - III in 1), and the test partitions (c) will be utilised to evaluate and exemplify a monitoring case (Step IV). The number of latent variables can be optionally validated after step II, through cross-validation.

The results will be compared to grid search, and other benchmark optimisation techniques such as the Genetic Algorithm (GA) and Nelder–Mead simplex, and state-of-the-art MSPC techniques, such as the variational autoencoder MSPC, the deep autoencoder MSPC and manifold learning MSPC. The success of the convergence parameters, both in the proposed method and the compared techniques, is evaluated with the Fault Detection Rate (FDR), which measures the percentage of faulty points successfully captured in the test dataset.

3. Results and discussion

Before going into the more complex kernel MSPC, the linear MSPC results are discussed in Appendix E, where it is showcased that while it can capture the **E1** fault in the control chart, the **D1** remains undiscovered, under the control limits.

3.1. Optimising K-MSPC with line search

In K-PCA, the explained variance of the PCs is dependent on the kernel parameter used. The training kernel matrix is a symmetric matrix that has as many rows and columns as the initial number of observations. This drives the maximum number of possible PCs to increase if the number of rows is larger than the number of variables, which is also the case here. In Fig. 2(a), the explained cumulative variation is shown. With an increase in the Gaussian kernel width, the explained variance by the first PCs increases.

When evaluating the kernel parameters through line-search, the easy-to-detect **E1** fault finds a convergence at a minimum. However, the more difficult fault oscillates around the same ρ_b value that the linear case resulted in. This can indicate that the evaluated range is

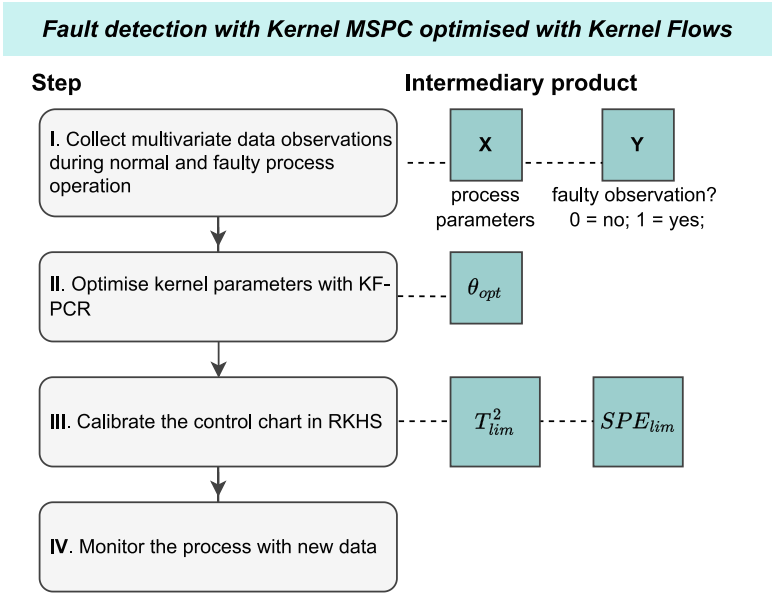


Fig. 1. Proposed fault detection workflow, using the ρ_a loss function.

insufficient, the line search discretisation is too coarse, or fitting a single kernel parameter to the entire dataset is not appropriate for the task. Thus, a more specialised technique that is able to select the kernel function and kernel parameters individually for each variable is needed.

3.2. Kernel MSPC optimised with the proposed method

To isolate the effect of each modelling choice, we first evaluate kernel MSPC with a single optimised kernel parameter. We then progressively introduce variable-wise kernel parameters and kernel-family learning to assess their incremental impact on fault detection performance. In the literature evaluated on kernel-MSPC, only one kernel parameter is fitted for the whole dataset $k(\mathbf{X}, \mathbf{X}, \theta)$. This scenario works well for the "easy to detect" faults — the ones that work well with the traditional MSPC. This is also the case for the **E1** fault.

Fig. 3(a) showcases the ρ_a loss, that is, minimising the percentage of misclassifications between healthy and faulty observations with K-PCR. It is noted that the loss converges to a minimum in relatively few iterations and is stabilised there. The loss is expected to approach a minimum, but not necessarily 0, since the model optimises a classification with a discriminant analysis, and not the control charts directly (like in ρ_b). As can be seen in Fig. 3(b), for the test case where the process starts in control and develops a fault after eight hours, the control chart correctly identifies the fault. The main contributors to the high-T^2 values are variables **x01**, **x17**, **x18**, **x43**, and **x49**.

However, in the more difficult cases, such as the **D1** case, fitting one kernel parameter for the whole dataset is not optimal. The fault **D1** cannot be identified by regular MSPC nor with single-kernel parameter optimisation techniques.

Another option would be to fit individual kernel parameters for each of the variables and sum the individual kernel matrices together. However, for this case study, doing so for 52 variables increases the complexity of the grid search, and the task becomes computationally expensive and time-consuming.

Optimising K-PCR with Kernel Flows to discriminate between the healthy and faulty states by fitting individual kernel parameters to each variable is a solution to the issue. As seen in Fig. 4(a), the loss converges to a minimum in less than 300 iterations, and, for the first time, the **D1** fault is captured in a control chart (Fig. 4(b)).

When fitting individual kernel parameters to each of the variables, not only do we obtain a better fault identification, but we also gain interpretability on the model. By evaluating the convergence of the kernel parameters, we can quantify the amount of variation that is included from each variable. As the Cauchy kernel is defined in Eq. (5), the smaller the kernel parameter, the less variation from that variable is included.

We emphasise that the learned variable-wise kernel parameters quantify model sensitivity rather than causal physical importance. In an additive kernel, each variable contributes to the similarity metric and therefore to the feature-space geometry used by K-PCA and K-PCR. Variables assigned larger effective influence are those whose deviations most improve discrimination between normal and faulty operation under the chosen loss. For faults that are not directly measured (e.g., **D** feed temperature), the method is expected to highlight propagated process responses in other sensors rather than the unmeasured root cause.

Appendix H presents visualisations on the convergence of kernel parameters, in the case where a Cauchy kernel has been utilised in (a) **Case 1** with individual kernel parameters for each length scale (σ_l), but common fixed amplitude parameter ($\gamma_l^2 = 1$) for each variable l, and (b) **Case 2** with learned amplitude parameter and length scale for each variable. In **Case 1**, the variables **x02** (A feed rate) and **x03** (D feed rate) converge in small σ_l values. They are early warning or fine-resolution indicators. The learned kernel assigns tight similarity to **x02** and **x03** variables, and even small departures from normal reduces the similarity quickly. This suggests that faults in the training data were best discriminated by subtle or fast deviations in these variables. An operator might treat **x02** and **x03** as sensitive variables, where small changes may already be diagnostically meaningful, and might investigate the usage of tighter control limits or sampling for these variables, and investigate small, persistent offsets. Given that the fault **D1** occurs in the **D** feed temperature, which is not measured amongst the variables, these flagged feed variables are consistent with the fault behaviour.

The variables **x20** (Stripper Steam Flow), **x44** (E Feed percentage), **x49** (Separator liquid flow) behave like coarse or shift indicators. For them, the learned kernel similarity decays slowly, meaning the model considers a wide range of values still consistent with the "normal", unless deviations become substantial. The variables contribute to fault detection through larger-scale shifts or level changes, sustained offsets rather than fine fluctuations. These variables are less useful for catching early deviations, but become informative when there is a pronounced change.

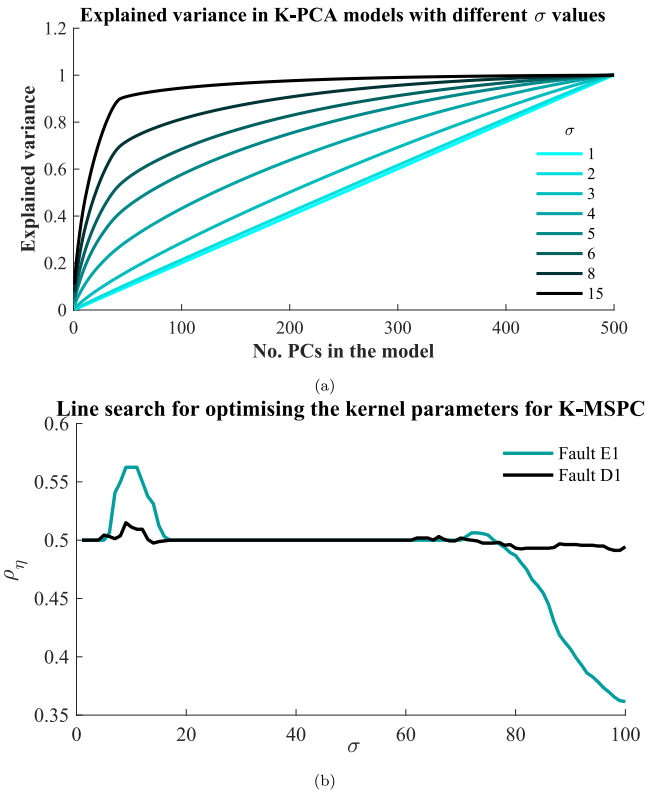


Fig. 2. (a) The variation explained by PCs in K-PCA is dependent on the shape of the kernel. In the showcased methods, the kernel function utilised is a Gaussian one. (b) The loss value (ρ_b) evaluated for line-search of the Gaussian kernel parameter, for both E1 and D1 faults.

When the amplitude parameter γ_l^2 is learned jointly with the scale parameter σ_l (**Case 2**) in the variable-wise Cauchy kernel, the model performs implicit relevance weighting across variables. Variables **x16** (Stripper level), **x19** (Stripper temperature), and **x50** (Stripper liquid flow) converge to large values of γ_l^2, indicating that these signals are primary contributors to discriminating between normal and faulty behaviour and therefore dominate the kernel similarity structure. In contrast, variables **x07**, **x13**, and **x20** converge to small values of γ_l^2, suggesting that their deviations are comparatively less informative, for instance due to redundancy with other variables or weak association with fault conditions. Simultaneous learning of σ_l further determines the deviation scale at which each variable becomes informative, thereby separating which variables matter (through γ_l^2) from what type of deviations matter (through σ_l).

The analysis focuses on a single test observation at index 200, corresponding to approximately two hours after fault onset. At this stage, the process has already deviated from normal operation, but transient and dynamic effects may still be present. Fig. 5(a) shows the variables identified through classical T^2 variable contributions, while 5(b) presents the variable-wise attributions obtained from the additive Cauchy kernel.

In Fig. 5(a), the dominant T^2 contributions are associated with variables related to separation and downstream units, including the product separator temperature and underflow (**x12**, **x18**), stripper temperature and steam-related variables (**x19**, **x20**), and several composition measurements in the reactor feed, purge, and product streams (**x27**, **x29**, **x30**, **x39**, **x41**). In addition, manipulated or flow-related variables such as feed composition ratios and valve positions (**x46**, **x48**, **x50**) exhibit elevated contributions. These results indicate a fault impact that propagates across material balances, separation efficiency, and downstream composition control.

The additive kernel attribution in Fig. 5(b) identifies a partially overlapping but broader set of influential variables. In addition to key variables already highlighted by the T^2 contributions, the kernel-based analysis assigns relevance to upstream feed rates and reactor input variables (**x03**, **x07**), separator level and purge-related compositions (**x13**, **x33**, **x37**), and product composition variables (**x40**). This reflects the ability of the additive kernel to capture distributed, nonlinear contributions across multiple units, including variables whose individual deviations may be moderate but collectively informative for fault discrimination.

It is important to emphasise that the presented results correspond to a single time point, and variation in variable contributions is expected across the fault trajectory. Not all variables identified here are necessarily persistent fault indicators; rather, they reflect the instantaneous process state two hours after fault development. Consequently, the identified variables should be interpreted as indicative of fault propagation pathways at this operating stage, rather than as an exhaustive or definitive fault signature.

3.3. Sensitivity studies for the proposed method

The reliability of the method is evaluated through sensitivity studies. First, the effect on the initial number of PCs. Fig. 6(a) showcases that, regardless of the initial number of PCs in the range 1–10, the convergence values after the outer-loop cross-validation evaluation converges in the same number of PCs. This convergence is also reflected when we are evaluating the ρ_b on the test set post-optimisation, in 6(b). What is notable is the fact that, for any number of initial PCs evaluated, almost all the faulty points from the dataset are captured.

When evaluating the sensitivity of the method with the learning rate (Fig. 7), we can observe three main things: (I) If the learning rate is

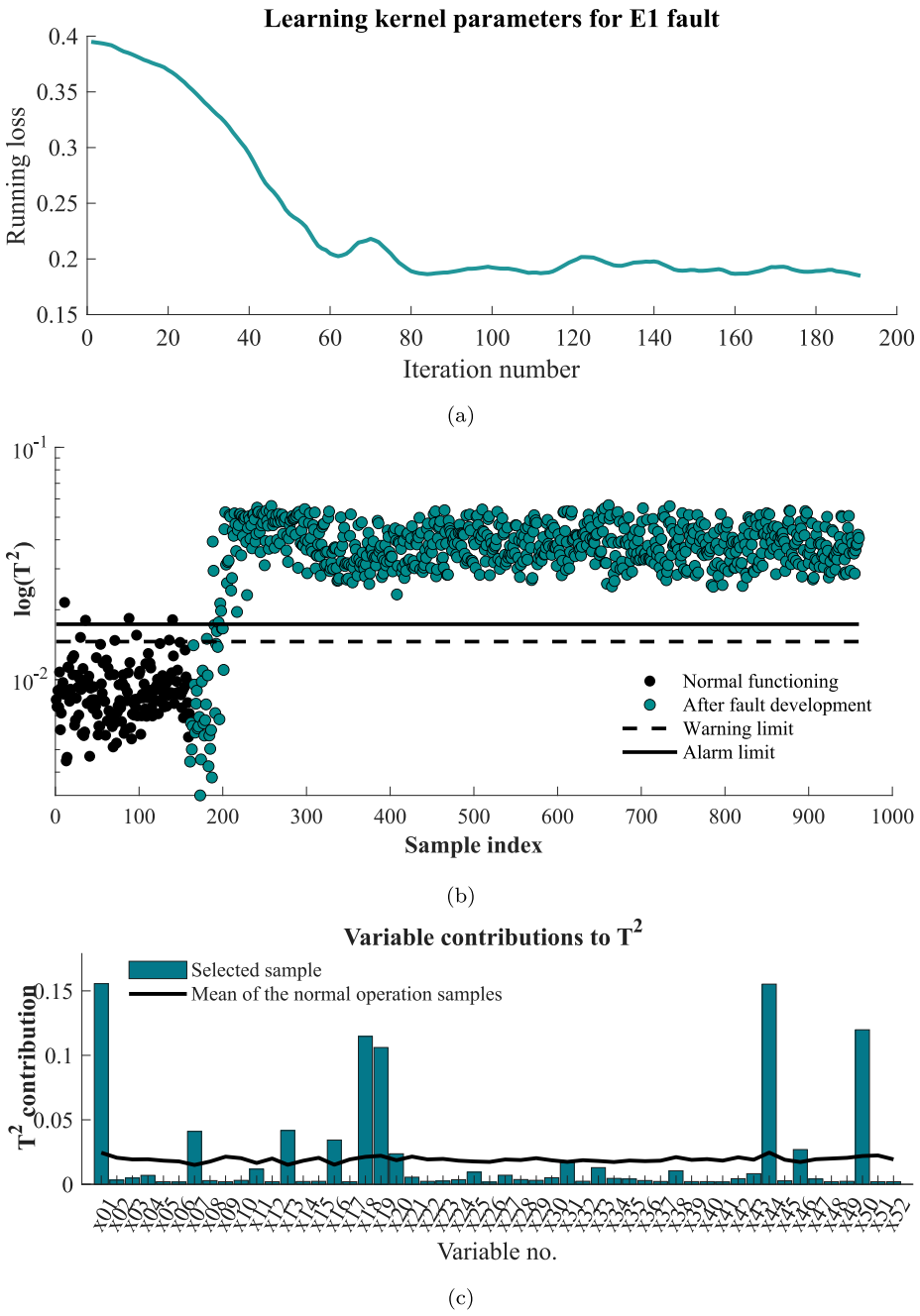


Fig. 3. (a) Single-kernel parameter learning for a Gaussian kernel, trained with the E1 fault and (b) the resulting T^2 control chart with optimised kernel parameters, along with the (c) variable contributions to the T^2 chart for the 240th sample.

too small, there is the risk of the unnecessarily extending the number of iterations, or the optimiser being stuck in a local minima; (II) If the learning rate is too big, there is the risk of the optimisation being too unstable; (III) For the evaluated options, the larger the learning rate, the faster the convergence.

Fig. 8(a) showcases the increase in optimisation time for individual kernel parameters learning for different proportions of samples per iteration subsampling. While the amount of time for running 200 iterations greatly increases in the case of the ρ_a loss, the ρ_b showcases a small increase in time as the amount of samples increase. The same results cannot be said for the performance of the optimized models. For both losses, almost all faulty points are detected, with minimal alarms on the test set (Fig. 8(b)).

Additional sensitivity studies on the seed effect are provided in Appendix F.

3.4. Calibration data needs

The previous scenarios consider the parameter tuning on the calibration partition that represents process data having the same fault as in the testing dataset. When calibrating the control charts, it is not always possible to capture all possible faults through experiment or simulation. Ideally, if a few faults have been simulated and considered, the model can catch the faults regardless if it has seen similar fault data in the calibration phase.

Fig. 9 presents the control charts resulting when the kernel parameters have been learned with **E1**+**D1** faulty process data, and the models have been applied to **E2** and **D2** process faults. The resulting control charts illustrate that the faults in the testing datasets **E2** and **D2** can still be captured.

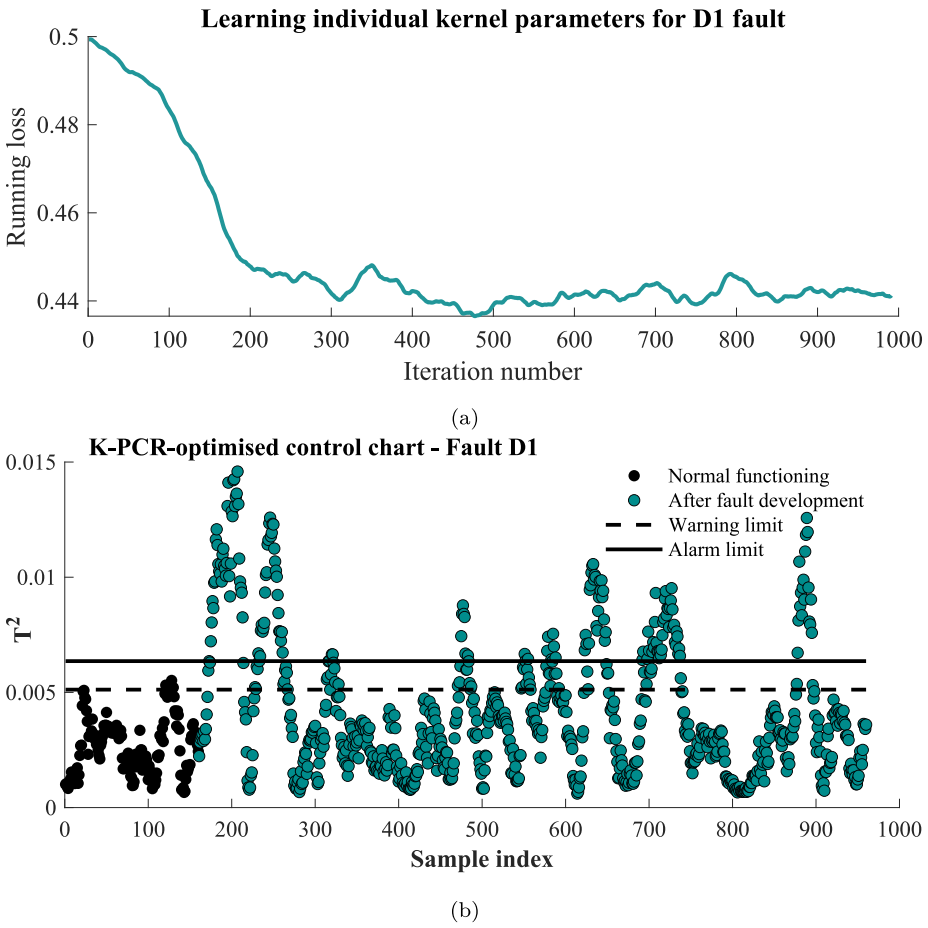


Fig. 4. (a) Kernel flow loss convergence for the D1 K-PCR optimisation with 4 PCs and a Cauchy kernel, and (b) the resulting control chart for the converged parameters.

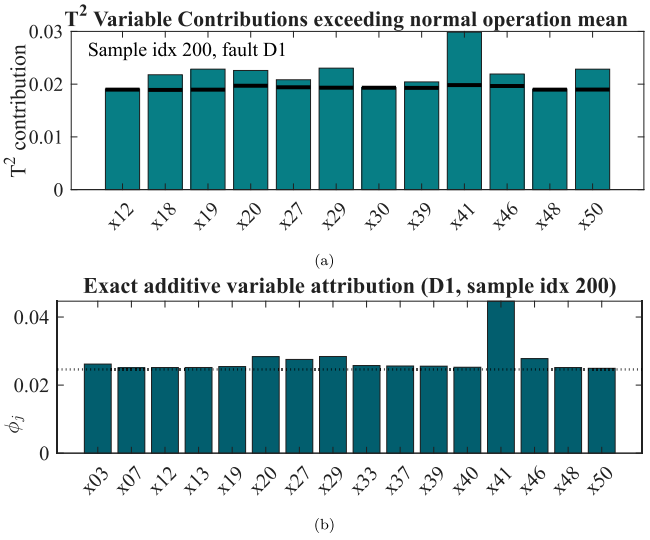


Fig. 5. (a) Variable contributions to the T^2 control chart for the 200th test sample (after fault development), in fault **D1**, along with (b) variable-wise K-PCR attribution for the classification model.

The results of the fault detection of fault **D2** do not seem impressive when compared to the easier-to-detect **E1** fault. However, compared to other optimisation methods, such as the Genetic Algorithm [20] and the Nelder–Mead Simplex optimisation utilising the same loss function ρ_1, the KF-based optimisation method performs better. The results of these methods are present in Appendix I.

One might ask how the proposed approach compares to out-of-the-box supervised classifiers trained on the same labelled data. The

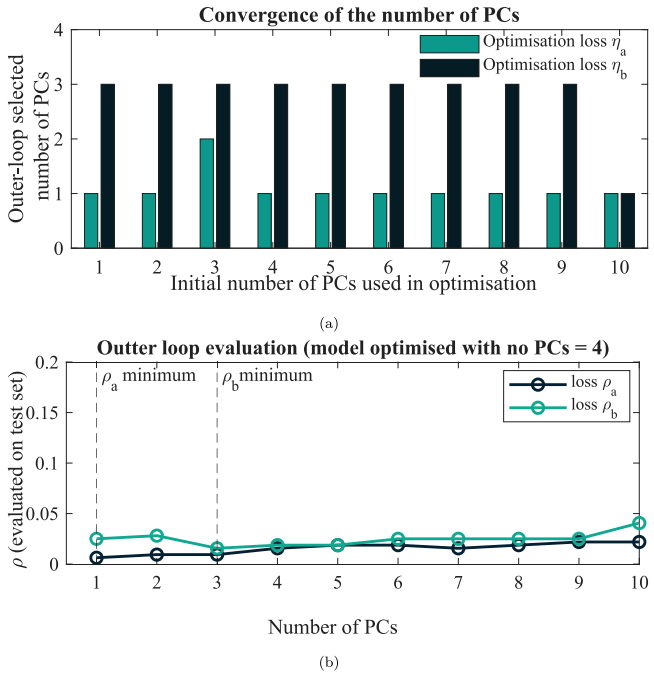


Fig. 6. Sensitivity study on the number of PCs, for fault **E1**, using a gaussian kernel, with individual kernel parameters for each variable. (a) showcases the convergence no. of PCs post-optimisation and (b) visualises the ρ_b metric on the test partition, post-optimisation.

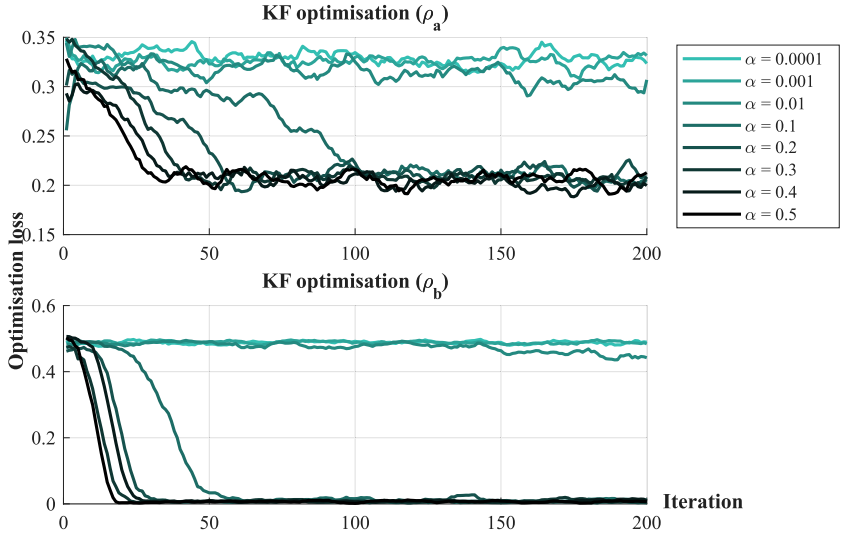


Fig. 7. Sensitivity study on the learning rate parameter using the **E1** fault, for a gaussian kernel, with individual kernel parameters fitted for each variable. A gradient clipping of 0.5 was employed.

corresponding results are summarised in Table 3 in terms of Fault Detection Rate (FDR) and False Alarm Rate (FAR). For the easy-to-detect fault **E1**, several classifiers achieve high FDR with low FAR. However, for the difficult-to-detect case **D1**, most classifiers exhibit substantially elevated false alarm rates, which is not observed for MSPC-based methods (including the proposed optimisation scheme as well as GA- and Nelder–Mead-optimised K-MSPC), where FAR remains near zero across faults.

The contrast becomes more pronounced in the cross-fault generalisation setting: when models are trained on a limited set of faults (**E1** and **D1**) and then evaluated on unseen faults (**E2** and **D2**). Even for the easy-to-detect **E2**, the classifiers tend to trigger frequent false alarms during normal operation (FAR between 15% and 48%), and many also underperform in FDR. For the difficult-to-detect **D2**, both FAR and FDR are typically close to chance level (around 50%), indicating limited transferability of these out-of-the-box classifiers in this monitoring setup. In contrast, the proposed MSPC-based approaches maintains low FAR while achieving detection.

The previous case investigated whether the fault can still be detected, even though prior information about the fault is not known. Further, we investigate the case where we do not know how the inner structure of the data looks (the shape of the kernel function). Assuming we do not know the structure of the data for the fault **E1** detection, we use a combination of kernels. In Fig. 3(b), the appropriateness for a Gaussian kernel to identify the fault is confirmed, with all the faults being captured using the kernel function. In Fig. 10(b), it can be observed that the Gaussian kernel was tuned to the highest amplitude, whereas the rest of the kernels converged at a significantly smaller value. The amplitude parameter for each kernel approaches 0 for the kernels that are not appropriate to represent the problem.

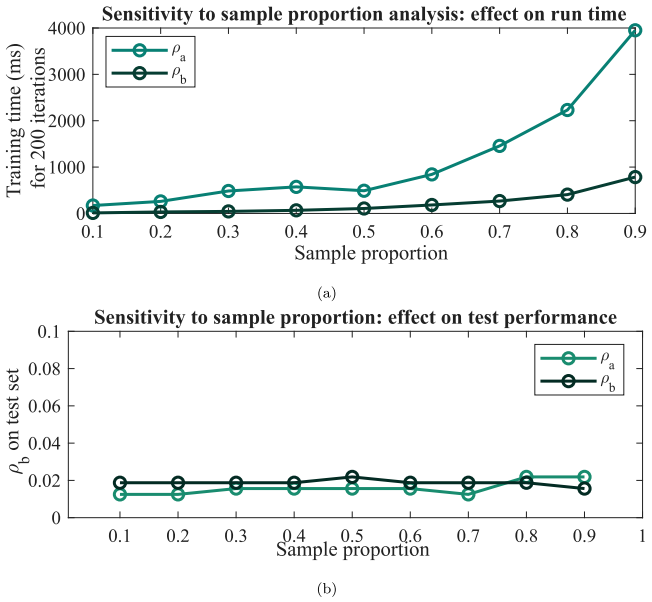
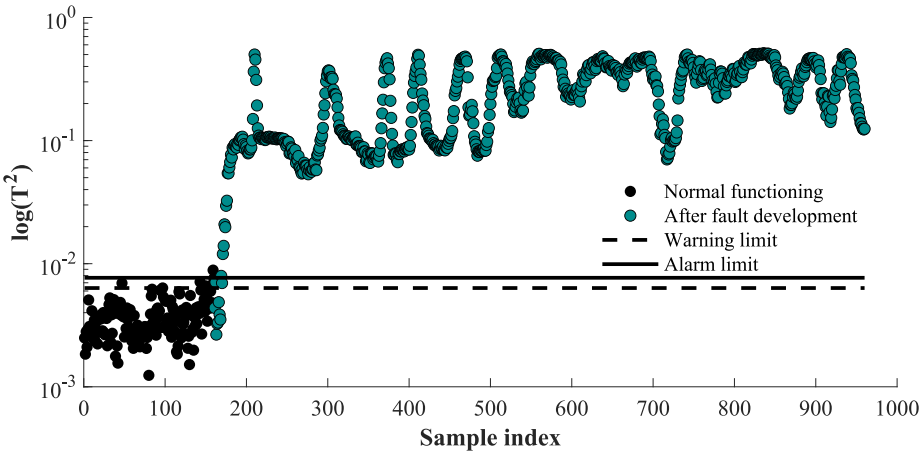
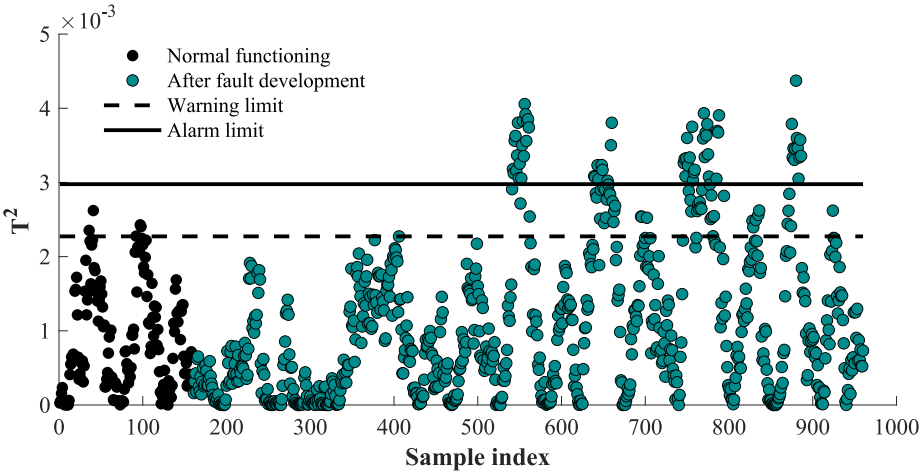


Fig. 8. Sensitivity study for the sample proportion per iteration subset, evaluated on Fault **E1**, using a gaussian kernel with individual kernel parameters for each variable. (a) showcases the time performance with the proportion of samples utilised, while (b) visualises the ρ_b metric on the test partition, post-optimisation.



(a) **E2** fault.



(b) **D2** fault.

Fig. 9. Parameters optimised with **E1**+**D1** faults, and new data projected from **E2** (a) and **D2** (b) faults.

Table 3
A total of 32 out-of-the-box-classifier models has been tested and optimised in MATLAB's Classification Learner app. Here are the results from the best performing models in each class, in the Fault Detection Rate (FDR) and False Alarm Rates (FAR). The optimal hyperparameter settings are: (1) fine tree: 100 splits, gini's diversity index split criterion; (2) Kernel SVM: automatic* number of expansion dimensions, automatic* regularisation strength and kernel scaling, 1000 iterations; (3) Cubic KNN: 10 neighbours, Minkowski distance metric, equal distance weight; (4) Ensemble subspace discriminant: 30 learners, 26 subspace dimensions; (5) Wide neural network: one fully connected layer, size of the 1st layer: 100, ReLU activation, 1000 iterations; (6) Medium tree: 20 splits, Gini's diversity index split criterion; (7) Quadratic SVM: quadratic kernel, automatic* kernel scale, box constraint level 1; (8) Medium KNN: 10 neighbours, euclidean distance metric, equal distance weight; (9) Ensemble bagged trees: bag ensemble method, decision tree learning type, 1919 max. splits, 30 learners; (10) Medium NN: one fully-connected layer, first layer size: 100, ReLU activation, 1000 iterations. (11) ρ_b loss, Gaussian kernel; (12) ρ_a loss, Cauchy kernel. *automatic = as optimised with MATLAB's Classification Learner.

Model	Train E1		Train D1		Train E1 + D1			
	Test E1		Test D1		Test E2		Test D2	
	FAR	FDR	FAR	FDR	FAR	FDR	FAR	FDR
Trees	0.003 (1)	0.996	0.223 (6)	0.461	0.172 (1)	0.906	0.450 (1)	0.62
SVM	0.000 (2)	0.994	0.096 (7)	0.296	0.400 (7)	0.640	0.538 (7)	0.531
KNN	0.000 (3)	0.991	0.254 (8)	0.485	0.488 (8)	0.971	0.525 (3)	0.532
Ensemble trees	0.000 (4)	0.990	0.189 (9)	0.589	0.156 (9)	0.320	0.625 (9)	0.720
Neural networks	0.000 (5)	1	0.272 (10)	0.505	0.456 (5)	0.065	0.488 (10)	0.531
Proposed method	0.003 (11)	0.993	0.000 (12)	0.214	0.001 (11)	0.992	0.000 (12)	0.190

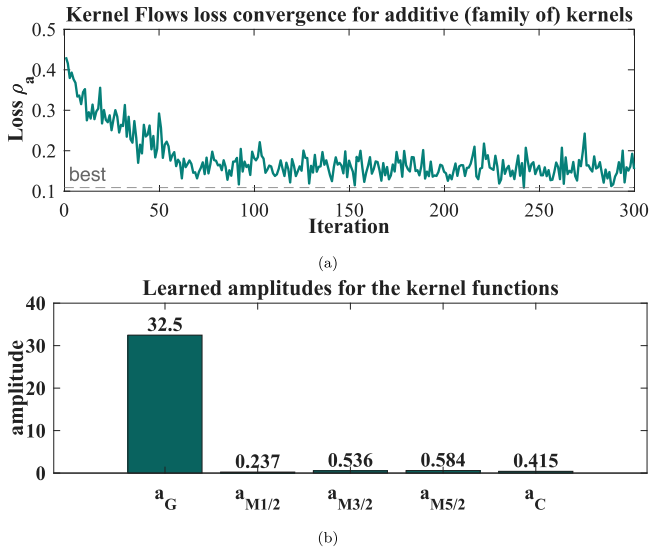


Fig. 10. Optimising a family of kernels for the **E1** case: (a) the convergence of the optimisation and (b) the fitted amplitudes for each kernel function.

3.5. Comparison with other benchmark K-MSPC optimisers and state-of-the-art MSPC methods

While only four faults, two easy-to-detect and two difficult-to-detect, have been considered for an in-depth analysis in the previous subsections, the original dataset has twenty-one simulated faults. The fault detection rate across all these faults is presented in Table 4, along with benchmarking in traditional PCA, K-PCA optimised with other optimisers, and other state-of-the-art procedures. From the presented results, the following can be observed: (i) a badly optimised kernel MSPC can have a worse performance than the traditional MSPC; (ii) overall, the proposed optimisation method overperforms both benchmark and state-of-the-art MSPC-based techniques; (iii) generally, the ρ_a-based optimisation overperforms ρ_b with approx. 3%; (iv) most of the times, ρ_a and ρ_b optimisations converge in the same kernel parameters.

While the methodology provides timely and performance benefits when using a single length scale for the parameters, for some of the faults, the performance can be farther improved by the usage of additive kernels for each variable. Fig. 11 showcases that faults **4**, **10**, **11**, **14**, **17** and **21** get significantly improved by the usage if the process variables are treated separately.

3.6. Further discussion on advantages and limitations

The Kernel-MSPC optimisation method presented in this paper has the following advantages when compared to the existing literature:

- I. *Difficult faults are captured by individual-parameter models.* It was illustrated in the difficult to catch faults (**D1**, **D2**), fitting the same kernel parameter for the whole dataset is not optimal. The fault cannot be caught in the control charts. Optimising a kernel

Table 4
Fault detection rate (FDR) in T^2 charts across methods for the Tennessee Eastman Process. The highest FDR for each fault is highlighted in bold. The number of latent variables (7) and the use of Gaussian kernel was kept constant between the methods.

Fault	PCA linear	K-PCA line search	K-PCA Nelder–Mead	K-PCA genetic algorithm	dAE deep autoenc.	vAE variational autoenc.	ML manifold learning	K-PCA KF ρ_a (ours)	K-PCA KF ρ_b (ours)
1	**0.993**	0.993	0.993	0.823	**0.995**	0.990	0.543	0.990	**0.993**
2	**0.983**	0.971	0.971	0.970	0.979	0.975	0.236	**0.985**	0.980
3	0.044	0.128	0.128	0.138	0.068	0.071	**0.290**	0.214	0.065
4	0.113	0.174	0.175	0.000	0.338	0.439	0.203	**0.503**	0.225
5	0.266	0.328	0.328	0.004	0.318	0.266	0.356	**0.380**	0.368
6	0.991	0.994	0.994	0.994	0.999	0.999	0.990	**1.000**	**1.000**
7	0.763	0.508	0.509	0.201	0.999	0.823	0.443	**1.000**	0.990
8	0.971	0.958	0.960	0.955	0.970	0.925	0.855	**0.980**	**0.980**
9	0.038	0.120	0.120	0.135	0.068	0.043	**0.248**	0.170	0.170
10	0.433	0.536	0.538	0.533	0.394	0.330	0.465	**0.559**	**0.559**
11	0.314	0.361	0.364	0.324	0.398	**0.466**	0.294	0.365	0.324
12	0.973	0.974	0.975	0.790	0.974	0.959	0.831	**0.976**	**0.976**
13	0.940	0.940	0.940	0.446	0.941	0.931	0.868	0.944	**0.946**
14	0.873	0.800	0.805	0.413	0.964	**0.986**	0.194	0.800	0.744
15	0.069	0.174	0.175	0.174	0.103	0.099	0.293	**0.304**	**0.304**
16	0.268	0.428	0.428	0.418	0.300	0.223	0.420	**0.459**	0.456
17	0.775	0.795	0.796	0.775	0.761	0.823	0.436	0.690	0.654
18	0.894	0.884	0.884	0.891	0.901	0.893	**0.918**	0.910	0.910
19	0.013	0.054	0.055	0.003	0.094	0.116	**0.130**	0.048	0.055
20	0.355	0.414	0.414	0.446	0.259	0.396	0.461	**0.586**	0.503
21	0.308	0.293	0.293	0.319	0.365	0.168	**0.388**	0.320	0.319
Mean	0.542	0.563	0.564	0.472	0.580	0.568	0.470	**0.628**	0.596
Time (min)	0.05	15	10	23	9	6.3	0.9	3	2.3

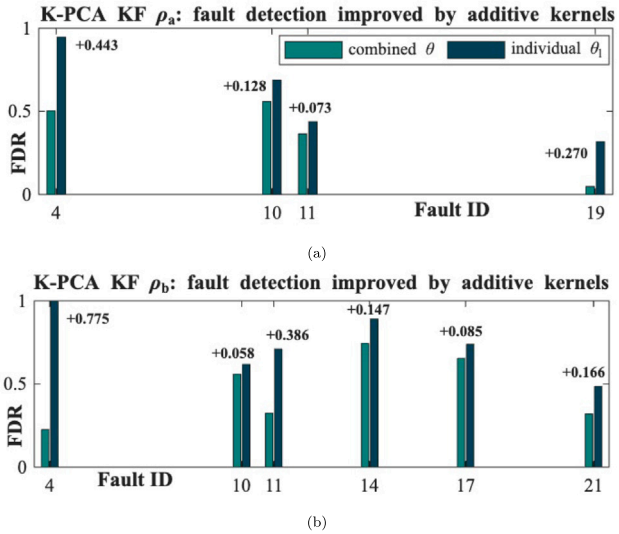


Fig. 11. Improvements in the FDR obtained by the usage of the additive kernels for each variable, when compared to the single-length models in Table 4, evaluated with (a) ρ_a loss and (b) ρ_b loss.

parameter for each variable proved to be a solution, but this is only feasible with an efficient optimiser such as the KF, as evaluating numerous parameters via grid search is computationally intensive. There are no instances in literature where individual kernels have been fitted for each of the variables in K-MSPC.

- II. *The proposed method is fast.* Even when optimising individual kernel parameters for each variable, the convergence was achieved fast, in a few hundred iterations and a maximum of four minutes, for a computer with the specifications mentioned in Appendix D.
- III. *The model is more interpretable.* By fitting individual parameters for each of the process variables, there are insights into the amount of variation utilised by the model form each individual variable. This can be an indicator of variable importance in fault discrimination and can aid in fault diagnostics.
- IV. *There is no need to know the optimal kernel function in advance*. If there is no prior knowledge of the optimal kernel, a combination of kernels can be utilised. In most of the existing literature, only the Gaussian kernel function is utilised, which is only applicable if the data is smooth. If data contains outliers, a Cauchy kernel is more suitable. If there is a need for a rougher kernel, a low-ν Matérn kernel can be utilised.

A fundamental limitation of the proposed method is that kernel parameter optimisation requires access to faulty process data. This moves kernel MSPC from a purely unsupervised framework towards a supervised or semi-supervised one, which contrasts with the traditional MSPC philosophy where only normal-operation data are used for model calibration.

As a result, the applicability of the proposed approach in industrial practice is conditional on the availability of representative fault data,

either from historical operation, controlled experiments, or process simulations. In industrial systems where faults are rare, unknown, or cannot be realistically simulated, the proposed method cannot be directly applied, and classical unsupervised MSPC remains the appropriate solution.

While digital twins and process simulators offer a potential means of generating fault data, their availability and fidelity are highly process-dependent. The proposed method therefore does not assume that such models are universally available, but rather that they constitute an enabling condition in specific industrial contexts.

While this study contributes to optimising the procedure for tuning the kernel MSPC parameters, an open research gap of utmost importance is the development of a fully unsupervised procedure for kernel calibration in K-MSPC. While the present method requires fault data to optimise the kernel parameters, a true unsupervised alternative would enable kernel adaptation solely from the healthy process data, without any knowledge of faulty conditions. Such a framework would preserve the spirit of traditional MSPC while retaining the nonlinear representational power of kernel methods. Developing and validating such an unsupervised kernel-learning approach would close an important methodological gap and make Kernel-MSPC a more general, self-calibrating framework suitable for real-world industrial deployment.

Another further improvement can reside in learning the kernel parameters in process phases, for a dynamic process that naturally undergoes shifts with time. While the current setup can utilise lagged variables, the training time increases with the variable size for ρ_a utilisation. The effect of utilising individual kernel parameters with lagged variables in analysed in Appendix J.

4. Conclusion

The aim of the present study was to propose an optimisation technique for K-MSPC. The proposed method is based on learning a K-PCR model that discriminates between the normal-operating and faulty-operating processes. The recently developed technique Kernel Flows has been utilised for the optimisation task. The optimised kernel parameters are then utilised in calibrating the K-MSPC control charts, resulting in better fault identification than the benchmark and state-of-the-art methods evaluated.

In K-MSPC, previous studies have proposed fitting a single kernel parameter for the whole process dataset. In the present study, we have demonstrated that fitting individual kernel parameters for each variable improves fault detection and adds to the interpretability of the model. The method has been tested with the benchmark Tennessee Eastman Process dataset and has shown success in detecting even the most difficult faults, including the ones that have not been captured by the original study of Russell et al. 2000. [24].

In a real-case scenario, data on all the possible faults is likely to be unavailable. The study demonstrated that calibrating the kernel parameters based on a monitored faulty operation can capture future unseen faults, eliminating the need for information in all possible faulty setups.

The development of a fully unsupervised kernel parameter learning strategy for kernel MSPC, capable of operating solely on normal-operation data, remains an open and important research direction.

CRediT authorship contribution statement

Zina-Sabrina Duma: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Victoria Jorry:** Writing – review & editing, Writing – original draft, Visualization, Validation, Investigation, Data curation. **Tuomas Sihvonen:** Formal analysis, Conceptualization. **Satu-Pia Reinikainen:** Supervision, Project administration, Formal analysis. **Lassi Roininen:** Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We acknowledge funding from the Research Council of Finland for the Centre of Excellence in Inverse Modelling and Imaging 2018–2025 (decision number 353095) and for the Flagship of Advanced Mathematics for Sensing, Imaging, and Modelling 2024–2031 (decision number 359183). VJ was supported through the Higher Education for Economic Transformation (HEET) program, funded by the World Bank through the Government of Tanzania.

Appendix A. Process variables

See Table A.5.

Appendix B. Background on K-PCA

Consider a dataset of n observations $\mathbf{x}_i \in \mathbb{R}^d$, $i = 1, \dots, n$, mapped into a reproducing kernel Hilbert space (RKHS) $\mathscr{H}$ through a feature map

$$\phi : \mathbb{R}^d \to \mathscr{H}.$$

In $\mathscr{H}$, the (empirical) covariance operator $C : \mathscr{H} \to \mathscr{H}$ is defined as

$$C = \frac{1}{n} \sum_{i=1}^{n} \phi(\mathbf{x}_i) \otimes \phi(\mathbf{x}_i), \tag{B.1}$$

where $\otimes$ denotes the outer product in $\mathscr{H}$. Although $\mathscr{H}$ may be infinite-dimensional, C is a well-defined linear operator.

Kernel PCA seeks eigenfunctions $v \in \mathscr{H}$ satisfying

$$Cv = \lambda v, \tag{B.2}$$

where $\lambda \geq 0$ denotes the associated eigenvalue.

By the representer theorem, all eigenfunctions of C lie in the span of the mapped training data, and can therefore be written as

$$v = \sum_{i=1}^{n} \alpha_i \, \phi(\mathbf{x}_i), \tag{B.3}$$

with coefficients $\alpha_i \in \mathbb{R}$.

Substituting Eq. (B.3) into Eq. (B.2) yields the finite-dimensional eigenvalue problem

$$n\lambda \boldsymbol{\alpha} = \mathbf{K}\boldsymbol{\alpha}, \tag{B.4}$$

where $\boldsymbol{\alpha} = [\alpha_1, \dots, \alpha_n]^\intercal$ and the kernel matrix $\mathbf{K} \in \mathbb{R}^{n \times n}$ is defined elementwise as

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle.$$

Thus, even though the covariance operator C may act in an infinite-dimensional space, the kernel PCA problem reduces to an eigenvalue decomposition of the finite kernel matrix $\mathbf{K}$.

To ensure zero-mean data in the RKHS, the kernel matrix is centred as

$$\mathbf{K}_c = \left(\mathbf{I} - \frac{1}{n} \mathbf{1}_n \mathbf{1}_n^\intercal \right) \mathbf{K} \left(\mathbf{I} - \frac{1}{n} \mathbf{1}_n \mathbf{1}_n^\intercal \right), \tag{B.5}$$

where $\mathbf{1}_n$ denotes the n-dimensional vector of ones.

The eigenvalue problem in Eq. (B.4) is solved using the centred kernel matrix $\mathbf{K}_c$. Eigenvectors are normalised such that $\lambda_h \boldsymbol{\alpha}_h^\intercal \boldsymbol{\alpha}_h = 1$.

Table A.5
Process variables.

Legend	Description	Legend	Description
x01	Time (h)	x28	Component E to Reactor (mol %)
x02	A Feed (kscmh)	x29	Component F to Reactor (mol %)
x03	D Feed (kg/h)	x30	Component A in Purge (mol %)
x04	E Feed (kg/h)	x31	Component B in Purge (mol %)
x05	A and C Feed (kscmh)	x32	Component C in Purge (mol %)
x06	Recycle Flow (kscmh)	x33	Component D in Purge (mol %)
x07	Reactor Feed Rate (kscmh)	x34	Component E in Purge (mol %)
x08	Reactor Pressure (kPa gauge)	x35	Component F in Purge (mol %)
x09	Reactor Level (%)	x36	Component G in Purge (mol %)
x10	Reactor Temperature (°C)	x37	Component H in Purge (mol %)
x11	Purge Rate (kscmh)	x38	Component D in Product (mol %)
x12	Product Sep Temp (°C)	x39	Component E in Product (mol %)
x13	Product Sep Level (%)	x40	Component F in Product (mol %)
x14	Product Sep Pressure (kPa gauge)	x41	Component G in Product (mol %)
x15	Product Sep Underflow (m^3/h)	x42	Component H in Product (mol %)
x16	Stripper Level (%)	x43	D feed (%)
x17	Stripper Pressure (kPa gauge)	x44	E Feed (%)
x18	Stripper Underflow (m^3/h)	x45	A Feed (%)
x19	Stripper Temp (°C)	x46	A and C Feed (%)
x20	Stripper Steam Flow (kg/h)	x47	Compressor recycle valve (%)
x21	Compressor Work (kW)	x48	Purge valve (%)
x22	Reactor Coolant Temp (°C)	x49	Separator liquid flow (%)
x23	Separator Coolant Temp (°C)	x50	Stripper liquid flow (%)
x24	Component A to Reactor (mol %)	x51	Stripper steam valve (%)
x25	Component B to Reactor (mol %)	x52	Reactor Coolant (%)
x26	Component C to Reactor (mol %)		
x27	Component D to Reactor (mol %)		

Given the normalised eigenvectors $\boldsymbol{\alpha}_h$, the kernel PCA score for a new observation $\mathbf{x}$ is obtained by projecting its mapped feature vector onto the corresponding eigenfunction:

$$t_h(\mathbf{x}) = \langle v_h, \phi(\mathbf{x}) \rangle = \sum_{i=1}^{n} \alpha_i^{(h)} \, k(\mathbf{x}_i, \mathbf{x}), \tag{B.6}$$

where $k(\cdot,\cdot)$ denotes the kernel function.

These kernel principal component scores are subsequently used to construct monitoring statistics such as Hotelling's T^2 and residual-based measures in kernel MSPC.

Appendix C. Background on MSPC

Hotelling's T^2 statistic is utilised to identify model outliers — measurements that have scores far from the model centre [30]. For the ith sample, it can be computed as follows:

$$T_i^2 = \frac{\mathbf{t}_i^{\intercal}(\mathbf{T}^{\intercal}\mathbf{T})^{-1}\mathbf{t}_i}{n-1} \tag{C.1}$$

where $\mathbf{t}_i$ is a vector containing the scores for the ith sample, $\mathbf{T}$ is the complete score matrix, and n is the total number of samples.

If the unmodelled variation is stored in the residual matrix $\mathbf{E} = \mathbf{X} - \mathbf{T}\mathbf{P}^{\intercal}$, the SPEx for a sample i can be expressed as:

$$\mathrm{SPEx}_i = \sum_{j=1}^{m} e_{i,j}^2 \tag{C.2}$$

where $e_{i,j}$ is an element of the residual matrix for the ith sample and jth variable.

For K-PCA, Hotelling's T^2 can be computed in a similar fashion. The squared prediction error (SPE_x) [35] is as follows:

$$\mathrm{SPEx} = \mathrm{k}(\mathbf{X},\mathbf{X}) - \mathrm{k}(\mathbf{X})^{\intercal}\mathbf{C}\mathrm{k}(\mathbf{X}), \tag{C.3}$$

where $\mathrm{k}(\mathbf{X}) = [\mathrm{k}(\mathbf{x}_1,\mathbf{X}),\ldots,\mathrm{k}(\mathbf{x}_n,\mathbf{X})]^{\intercal}$ and $\mathbf{C}$ is the covariance matrix expressed in Eq. (??).

The confidence limits for the T^2 index can be computed as a function of the F-distribution

$$T^2_{\mathrm{lim},\alpha} = \frac{h(n-1)}{n-h} F_{h,n-h,\alpha}, \tag{C.4}$$

where n is the number of samples and h is the number of PCs. Another way to compute the indicators is by assuming a Gaussian distribution, where the warning limit (95%C.I.) is computed as

$$T^2_{\mathrm{warning}} = \mu_{T^2} + 2\sigma_{T^2}, \tag{C.5}$$

and the alarm limit (99%C.I.) is computed as

$$T^2_{\mathrm{alarm}} = \mu_{T^2} + 3\sigma_{T^2}. \tag{C.6}$$

The confidence limits for the SPE_x are expressed as a function of the χ^2-distribution

$$\mathrm{SPE}_{x_{\mathrm{lim}}} = \frac{\sigma^2_{\mathrm{SPE}_x}}{2\mu_{\mathrm{SPE}_x}} \chi^2_l \tag{C.7}$$

where $l = \frac{2\mu^2_{\mathrm{SPE}_x}}{\sigma^2_{\mathrm{SPE}_x}}$, μ_{SPE_x} is the SPE_x mean and $\sigma^2_{\mathrm{SPE}_x}$ is here the SPE_x variance.

Appendix D. Computational resources

The simulations have been ran on a **MacBook Pro** with an Apple M2 chip, 16 GB/RAM. The scripts were coded in Julia using Visual Studio Code.

Appendix E. Traditional non-linear MSPC results

In the regular PCA, if we consider the explained variance of the PCs (Fig. E.12), we can observe that there are no dominant variational profiles in the dataset. The cumulative explained variance of the first two PCs sums up to about 20% of the variation, and the increase in variance explained after 4 PCs is very shallow. The increase in model complexity does not justify adding more PCs. This is the first clue that suggests a non-linear approach might be optimal for the dataset.

In Fig. E.13(a), one can observe the calibration of the T^2 control chart with samples from a regular process functioning. The points represent Hotelling's T^2 scores, which express how well a data point fits in the model, in this case, in a 4 PCs PCA model. The closer a data point is to the model's origin, the lower the T^2 score is. It is, thus, a measure of consistency with the variation included in the model. Warning (95% CI limits) and alarm (99% CI limits) are calculated as in Eqs. (C.5) and (C.6). It is expected that 5% normal functioning samples are found outside the control chart limits for the 95% C.I., and respectively 1% for the 99% C.I. These are denoted "false alarms", and their minimisation is desired.

The SPE_x control charts represent the variation not included in the PCA model. When a process goes out of control, the SPE_x control

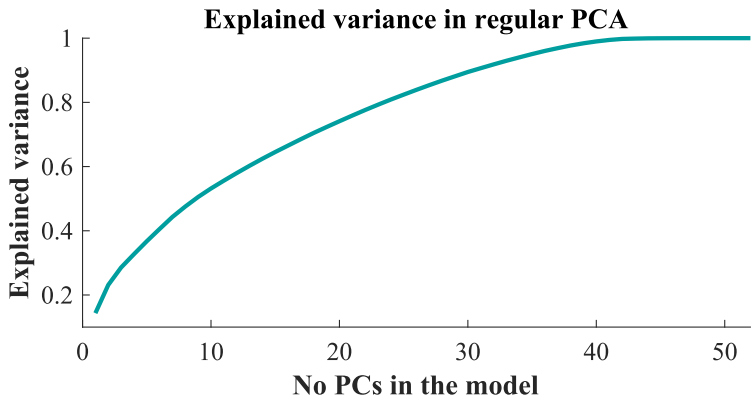


Fig. E.12. Explained variance in the regular PCA model of the normal-functioning data.

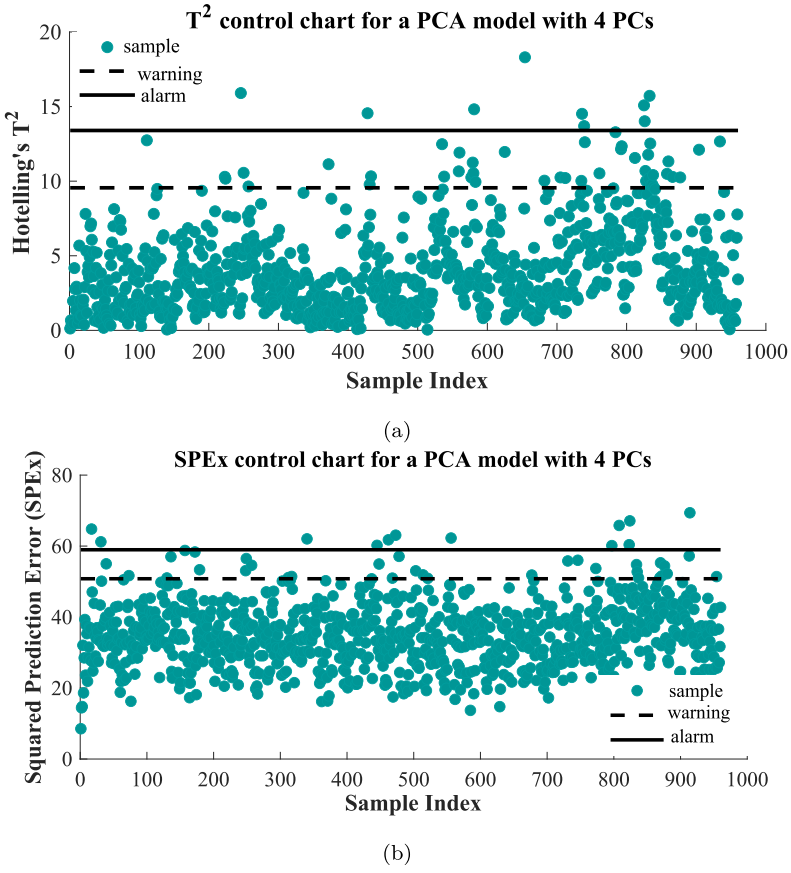


Fig. E.13. The T^2 and SPE_x control chart calibration with regular PCA and normal-functioning process data.

chart usually signals it first. The control chart can be visualised in Fig. E.13(b). The limits are computed as in Eq. (C.7).

Fig. E.14(a) and E.14(c) present scenarios where the control charts successfully identify the fault both in the included variance and the new variation presented by the faulty data. This fault (**E1**) can be successfully identified using traditional MSPC charts because it presents deviations from the modelled profiles. However, the more difficult fault (**D1**) is not successfully identified by the control charts. As seen in Figs. E.14(b) and E.14(d), the fault is hardly identified. Increasing the number of PCs in the model does not improve the control charts either. Fig. E.14(e) showcases the identification of the fault in the control chart. A constant 0.5 ρ value shows that regardless of the PCs included in the model, not only the normal-functioning 8 h sampling window is in-control lines, but also the fault that develops after 8 h. Thus, a linear approach is not appropriate for identifying the **D1** fault.

Appendix F. Convergence stability across random seeds

The proposed optimisation procedure relies on stochastic subsampling and random initialisation, which may in principle introduce variability in convergence behaviour. To assess the robustness of the method, we repeated the optimisation across multiple random seeds while keeping all other parameters fixed.

Specifically, we considered fault **D1** with 4 PCs, a Gaussian individual-variable kernel, learning rate 0.3, sampling proportion $sp = 0.4$, and 200 optimisation iterations. The experiment was repeated for both optimisation objectives, ρ_a (classification-based loss) and ρ_b (monitoring-based loss), using 20 different random seeds.

Fig. F.15 shows the mean and standard deviation of the optimisation loss trajectories across seeds. The results demonstrate that the optimisation converges consistently, with only minor variability between

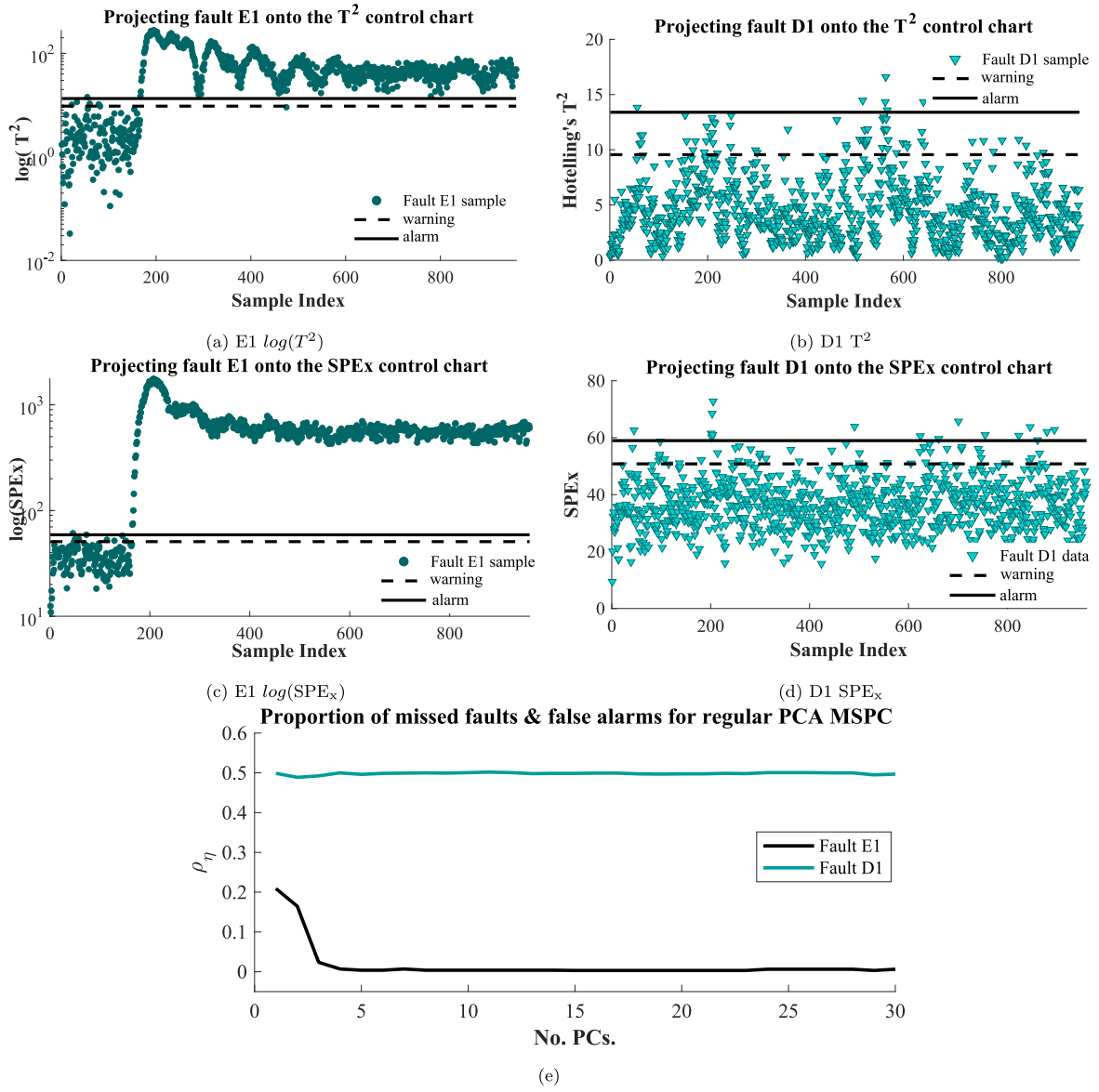


Fig. E.14. Capturing the faults using the regular PCA control charts. While the "easy" E1 fault is captured in both T^2 (a) SPE_x (d) control charts, the "difficult" fault is not captured in any of them. Increasing the number of PCs in the model (e) is not helpful in fault identification.

runs. No unstable or divergent behaviour was observed, indicating that the proposed Kernel-Flows-based optimisation is robust to random initialisation and stochastic sampling effects.

Appendix G. Benchmark and state-of-the-art MSPC methods utilised in comparison

When K-PCA was optimised with Nelder–Mead Simplex and Genetic Algorithm instead of Kernel Flows, loss ρ_b was optimised. Optimising with K-PCR loss (ρ_a) was also tested, but the classification loss was not appropriate to use with these optimisers, yielding very poor results, with the optimisers getting stuck very close to the initial value.

A sensitivity analysis was conducted for the genetic algorithm optimisation, varying the population size, maximum number of generations, elite count, crossover fraction, mutation operator, and function tolerance. The results in Table 4 are computed with the hyperparameters in Table G.6

For Nelder–Mead, the evaluated full-factorial design hyperparameters in Table G.7 showed very small differences in the results. Same as in the case for the Genetic Algorithm optimisation ρ_b could have been used, while ρ_a failed to learn a kernel that can distinguish between the faults.

A Deep Autoencoder (dAE) consists of an encoder, a decoder, and many hidden layers. A complete description of its architecture can be found in [36,37]. The structure uses an autoencoder neural network to learn a compressed representation of normal process data. Then it identifies anomalies based on high reconstruction errors or deviations in the latent space [38]. dAEs in MSPC have shown great promise in improving anomaly detection, particularly in high-dimensional process data. An application example of the TEP industrial process data is presented in [39]. In this work, Xiao et al. 2023 [39], proposed a deep autoencoder-based approach for MSPC, where the automatic variable selection based on reconstruction error from the autoencoder was used as a measure for fault detection. This work demonstrated that dAEs can

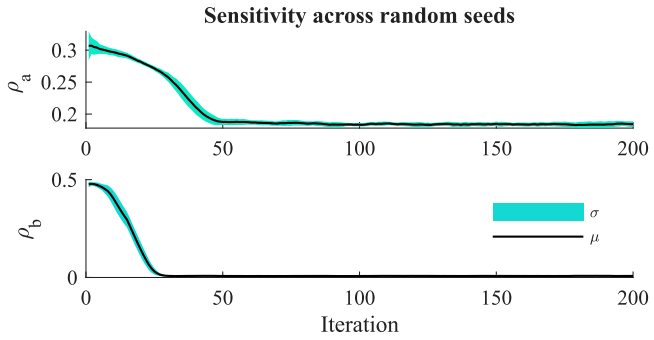


Fig. F.15. Mean and standard deviation of the optimisation loss across multiple random seeds for fault 4 and four latent variables. Results are shown for both ρ_a and ρ_b optimisation objectives, demonstrating stable and consistent convergence behaviour.

Table G.6
Genetic algorithm hyperparameters used for kernel parameter optimisation to compute the evaluations in Table 4.

Hyperparameter	Value
Population size	20
Maximum number of generations	15
Elite count	2
Crossover fraction	0.8
Mutation operator	Adaptive feasible
Function tolerance	10^{-4}
Initial population range	Data-driven (natural-log scale)
Random seed	Fixed (`rng(1)`)

Table G.7
Hyperparameters tested for the Nelder–Mead simplex optimisation.

Hyperparameter	Tested values
`TolX`	$\{10^{-6}, 10^{-4}, 10^{-2}\}$
`TolFun`	$\{10^{-6}, 10^{-4}, 10^{-2}\}$
`MaxIter`	{100, 300, 600}
`MaxFunEvals`	{200, 600, 1200}
`FunValCheck`	Enabled (fixed)
`Display`	Off (fixed)

Algorithm 2 Obtaining the FDR via dAE MSPC in Table 4

Require: Healthy data X_h, faults $\{X_f\}$
1: Normalize X_h with z-score $\rightarrow \hat{X}_h$
2: Train deep autoencoder (enc, dec) on $\hat{X}_h$
3: **for** each fault dataset X_f **do**
4: Normalize X_f using X_h's statistics
5: Encode $\hat{X}_f \rightarrow Z_f$
6: Reconstruct $\hat{X}_f \rightarrow \hat{X}_f^{rec}$
7: Compute T^2 in latent space
8: Raise alarms when statistics exceed control limits
9: Record detection rate
10: **end for**

capture complex, non-linear relationships between variables, resulting in higher fault detection rates and improved sensitivity and specificity in anomaly detection for industrial processes.

A symmetric multilayer perceptron autoencoder was used. The encoder consists of two hidden layers with ReLU activations followed by a linear latent layer, and the decoder mirrors the encoder with two ReLU layers and a linear output layer. The latent dimension was set to $k = 7$, and the hidden layer widths were (32, 16). The autoencoder was trained by minimising mean squared reconstruction error (MSE) on healthy data. Training was performed for 50 epochs with batch size 256 and learning rate 10^{-3} using stochastic gradient descent. Unless stated otherwise, the random seed was fixed to 1234.

The T^2 statistic was computed as the Mahalanobis distance of the latent code $\mathbf{z}$ (encoder output) using the covariance estimated from healthy latent codes. The SPE statistic was defined as the squared reconstruction error in the original variable space. Control limits were calibrated from healthy statistics using mean $+3\sigma$.

The dAE hyperparameters include: (a) Latent dimension: $k = 7$; (b) Hidden layers: (32, 16) with ReLU; (c) Epochs: 50; (d) Batch size: 256; (e) Learning rate: 10^{-3}; (f) Limits: mean $+3\sigma$ on healthy statistics.

A variational autoencoder (vAE) is a deep generative model with nonlinear feature extraction capabilities [40]. It works by training on normal process data to learn its underlying distribution in a lower-dimensional latent space. During operation, the vAE encodes new data and a monitoring index, often based on the reconstruction error or the deviation in the latent space, and detects deviations from normal behaviour. For fault diagnosis, methods such as probabilistic contribution analysis or fault isolation are used to identify the specific variables contributing to the fault, enabling root-cause analysis and improved process control. Different variants of vAE architectures to address particular challenges in process control have been proposed. For example, [41,42] have integrated vAE with LSTM networks to process time-series data and capture temporal dependencies.

The encoder consists of two fully connected hidden layers with ReLU activations, followed by two linear heads that output the latent mean and log-variance. The decoder mirrors the encoder with two ReLU hidden layers and a linear output layer. The latent dimension was set to $k = 7$.

The model was trained by minimising the reconstruction mean squared error plus a β-weighted KL divergence to a standard normal prior. We used $\beta = 1.0$.

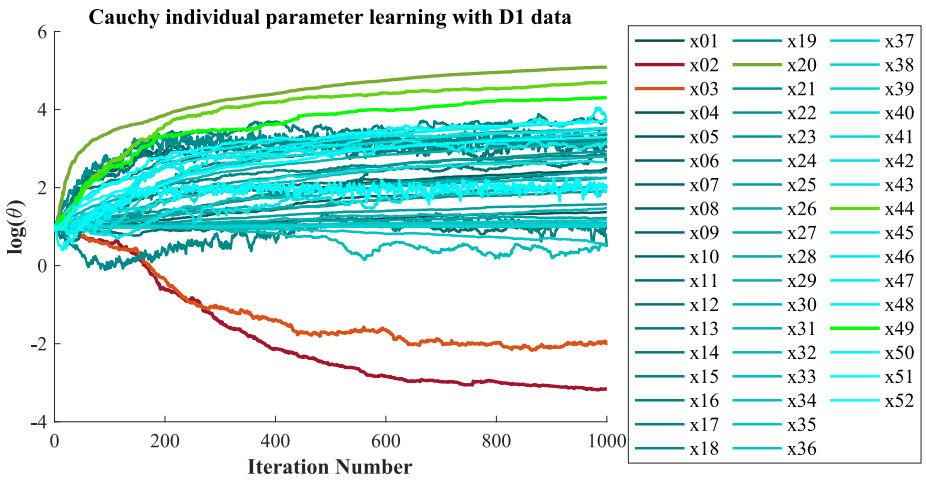
We trained for 80 epochs with batch size 256, learning rate 10^{-3} using stochastic gradient descent. Unless stated otherwise, the random seed was fixed to 1234.

The T^2 statistic was computed as the Mahalanobis distance of the encoder mean $\mu(\mathbf{z} \mid \mathbf{x})$ in latent space, using covariance estimated from healthy data. The SPE statistic was defined as the squared reconstruction error in the original variable space. Control limits were calibrated from healthy statistics using mean $+3\sigma$.

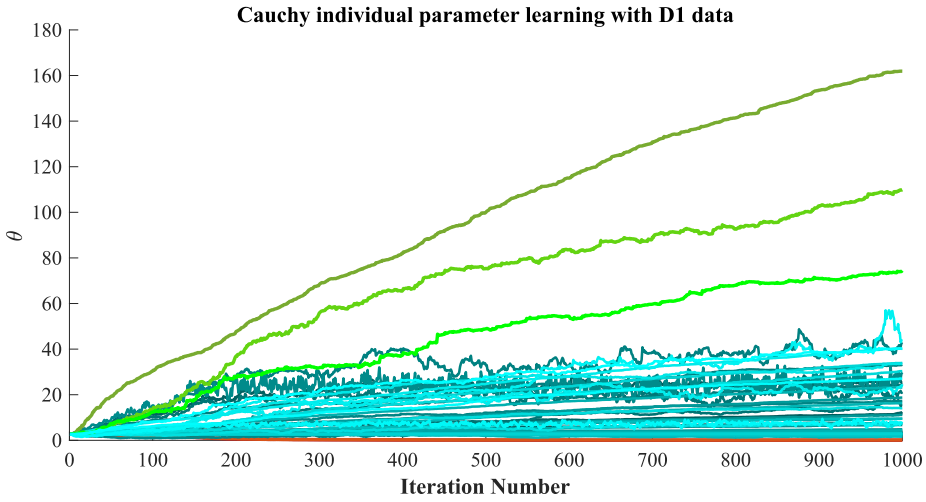
The vAE hyperparameters include: (a) Latent dimension: $k = 7$; (b) Hidden layers: (64, 32) with ReLU (c) Epochs: 80; Batch size: 256; (d) Learning rate: 10^{-3}; (e) $\beta = 1.0$; (f) Limits: mean $+3\sigma$ on healthy statistics.

Manifold learning (ML) enables the visualisation of complex processes, the identification of hidden fault signatures, and the creation of more effective monitoring strategies by projecting data onto a lower-dimensional manifold [43].

Manifold methods, such as Locally Linear Embedding (LLE), Isometric mapping (Isomap), and t-distributed Stochastic Neighbour Embedding (t-SNE), are standard, allowing for a better understanding of complex, dynamic processes where traditional linear methods, such as PCA, fall short [44]. Recently, manifold regularisation algorithms



(a)



(b)

Fig. H.16. The convergence of individual kernel parameters for the Cauchy kernel in the case of **D1** case learning, where $\theta = \{\sigma\}$ and $\gamma^2 = 1$ is kept constant. The case where $\theta = \{\sigma, \gamma^2\}$ is optimised is found in Appendix C.

Algorithm 3 Obtaining the FDR via vAE MSPC in Table 4

Require: Healthy data X_h, faults $\{X_f\}$
1: Normalize X_h with z-score $\rightarrow \hat{X}_h$
2: Train VAE $(\text{enc}_\phi, \text{dec}_\theta)$ on $\hat{X}_h$
3: **for** each fault dataset X_f **do**
4: Normalize X_f using X_h's statistics
5: Encode to latent mean μ, variance σ^2
6: Sample $z \sim \mathcal{N}(\mu, \sigma^2)$
7: Reconstruct $\hat{X}_f \rightarrow \hat{X}_f^{rec}$
8: T^2 in latent space
9: Raise alarms when statistics exceed control limits
10: Record detection rate
11: **end for**

Algorithm 4 Obtaining the FDR via ML MSPC in Table 4

Require: Healthy data X_h, faults $\{X_f\}$, neighbors k
1: Normalize X_h with z-score $\rightarrow \hat{X}_h$
2: Learn low-dimensional embedding via manifold learning (here Isomap)
3: **for** each fault dataset X_f **do**
4: Normalize X_f using X_h's statistics
5: Embed $\hat{X}_f$ in learned manifold
6: T^2 statistics
7: Raise alarms when statistics exceed control limits
8: Record detection rate
9: **end for**

have been intensively proposed for capturing the intrinsic geometrical structure in the given data. For example, in the manifold regularised stacked autoencoder (MRSAE) model proposed by [45], three graph regularisers, local and global variance, and local intrinsic graph will be embedded to learn the inherent structure of the given data.

A diffusion maps embedding was used to obtain nonlinear coordinates, with α-normalisation and out-of-sample extension via Nyström approximation. The embedding dimension was set to $k_{\text{embed}} = 7$ and $\alpha = 1.0$. The RBF kernel scale was selected using a median-distance heuristic.

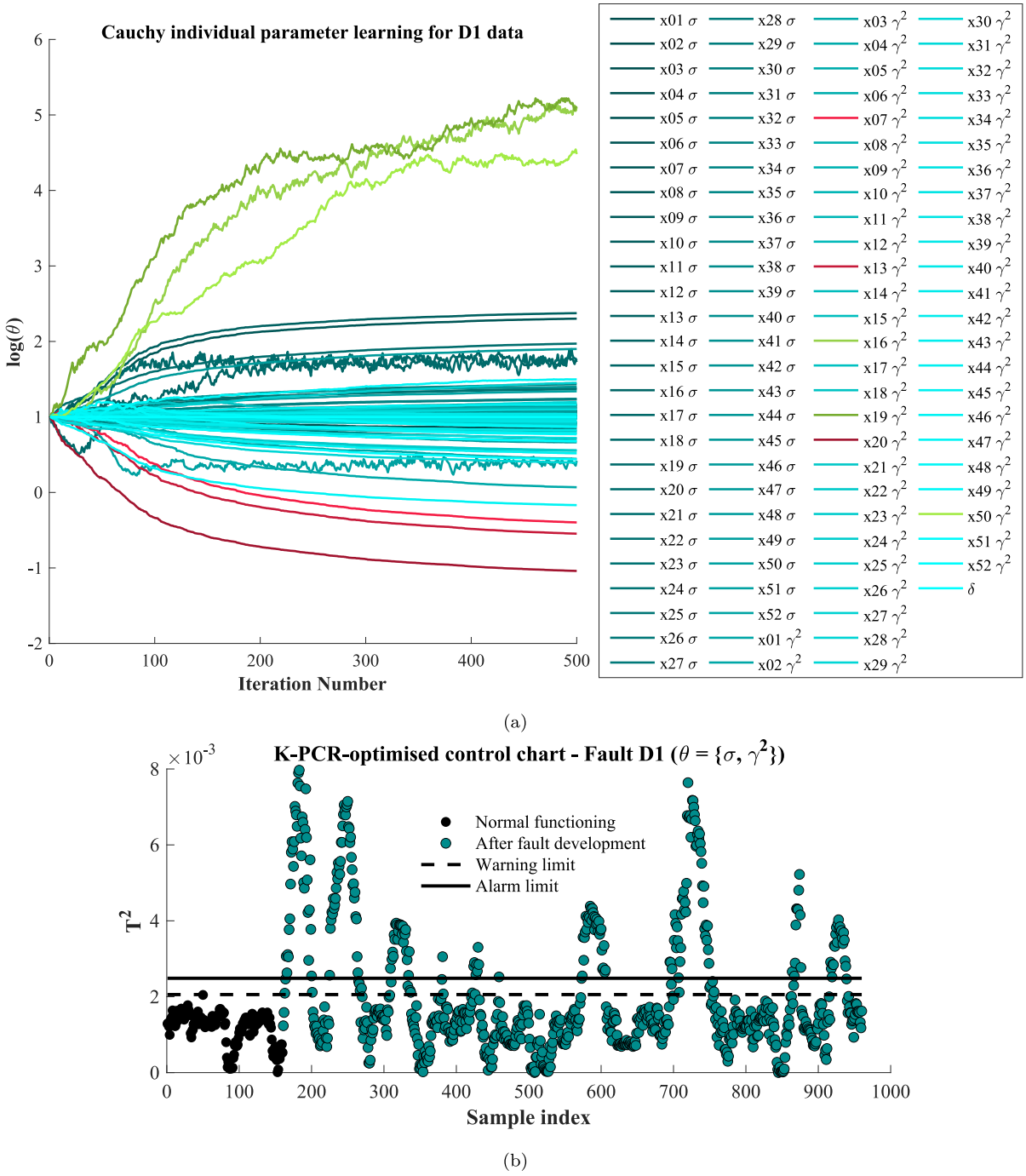


Fig. H.17. Individual parameter learning for a Cauchy kernel with $\boldsymbol{\theta} = \{\boldsymbol{\sigma}, \boldsymbol{\gamma}\}$ (a), and the resulting control chart (b).

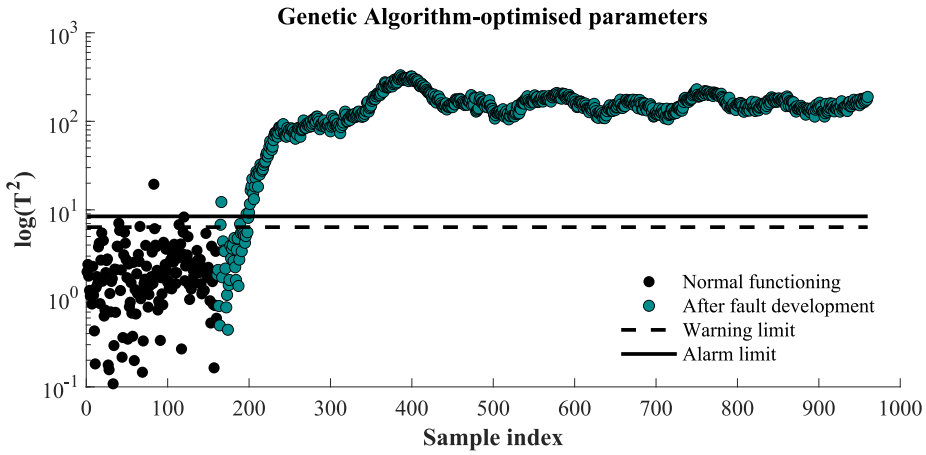
For the SPE statistic, local linear reconstruction was performed using LLE-style weights computed on k-nearest neighbours, with neighbourhood size 10. The T^2 statistic was computed as the Mahalanobis distance in the diffusion coordinates (covariance estimated from healthy data). The Q statistic was the squared local reconstruction error. Control limits were calibrated using mean $+3\sigma$ on healthy statistics.

Manifold Learning parameters: (a) Embedding method: diffusion maps (RBF kernel); (b) Embedding dimension: $k_{\text{embed}} = 7$; (c) Diffusion normalisation: $\alpha = 1.0$; (d) Kernel scale: median-distance heuristic; (e) Neighbourhood size for reconstruction: 10; (f) Limits: mean $+3\sigma$ on healthy statistics.

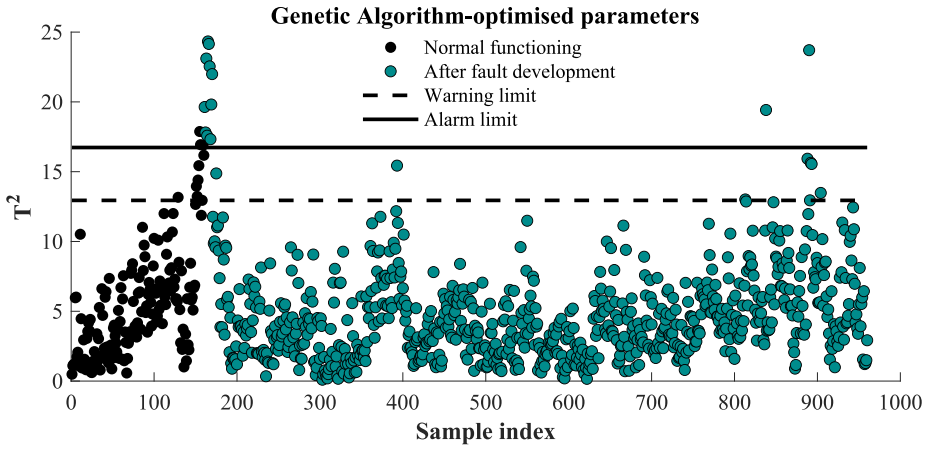
Appendix H. Optimising both the scalar and the kernel width for additive kernels

Fig. H.16 presents the convergence of kernel parameters in the case where $\boldsymbol{\theta}_l = \{\gamma_l^2 = 1, \sigma_l\}$. Variables **X02** and **x03** converge in small σ_l values, and variable **x20**, **x44**, **x49** converge in large σ_l values.

As observed in Fig. H.17(a), the scalar parameters (γ^2) of variables **x19**, **x16**, and **x50** converged in the highest values. They represent the stripper temperature, the stripper level and the stripper liquid flow. While there is no measured variable for the property driving the fault (the D feed temperature), the effects of this fault are indirectly observed



(a) **E2** fault



(b) **D2** fault

Fig. I.18. The control charts resulted by parameter optimisation with Genetic Algorithm, following the procedure and cost function from Jia et al. [20]. The optimal kernel function is the Gaussian kernel.

in the stripper's abnormal operation. Amongst the variables that got the lowest scalar values (**x20**, **x13**, and **x07**), there is the stripper steam flow, the product sep. level, and the reactor feed rate.

The cost of fitting 52 more parameters comes with a slight improvement in the control chart, which has no false warnings and a good capture rate for the fault, as seen in H.17(b).

Appendix I. Comparison with other optimisation methods and out-of-the-box classifiers

The present section presents results for the case where the kernel parameters are optimised with the **E1** and **D1** faults, and new faults are projected onto the control charts (**E2** and **D2**). The Kernel-Flows results for these are presented in Section 3.2, and this extension aims to present results with other benchmark optimisers for the same scenarios.

For the easy-to-detect fault **E1**, both the Genetic Algorithm (Fig. I.18(a)) and the Nelder–Mead Simplex optimisers (Fig. I.19(a)) have converged in similar kernel parameters, which make the control charts almost identical.

For the difficult-to-detect case (**D2**), however, the difference between the two benchmark optimisers increases. In the case of the Genetic Algorithm optimisation presented in Fig. I.18(b), the MSPC captures a few measurements after the fault development, and then the process appears to be in control. The Nelder–Mead simplex optimiser is not able to fit parameters so that the fault is captured, as observed in I.19(b). The KF-optimised K-PCR method introduced in the present paper outperforms both benchmark optimisers.

Appendix J. Usage of lagged variables

In continuous process monitoring, utilization of lagged variables to extend the original predictors list is possible. While there were no observed improvements in the fault detection rate for the faults **E1, E2, D1, D2**, there was an observed effect. As seen in Fig. J.20, the kernel parameters converged in an early convergence setting (50 iterations), when the variable lags were increased to 3, for the case where individual kernel parameters were fitted for each process variable.

Data and code availability

The data is publicly available [24]. The developed toolbox has been made publicly available in GitHub at https://github.com/sab-in-science/k-mspc.

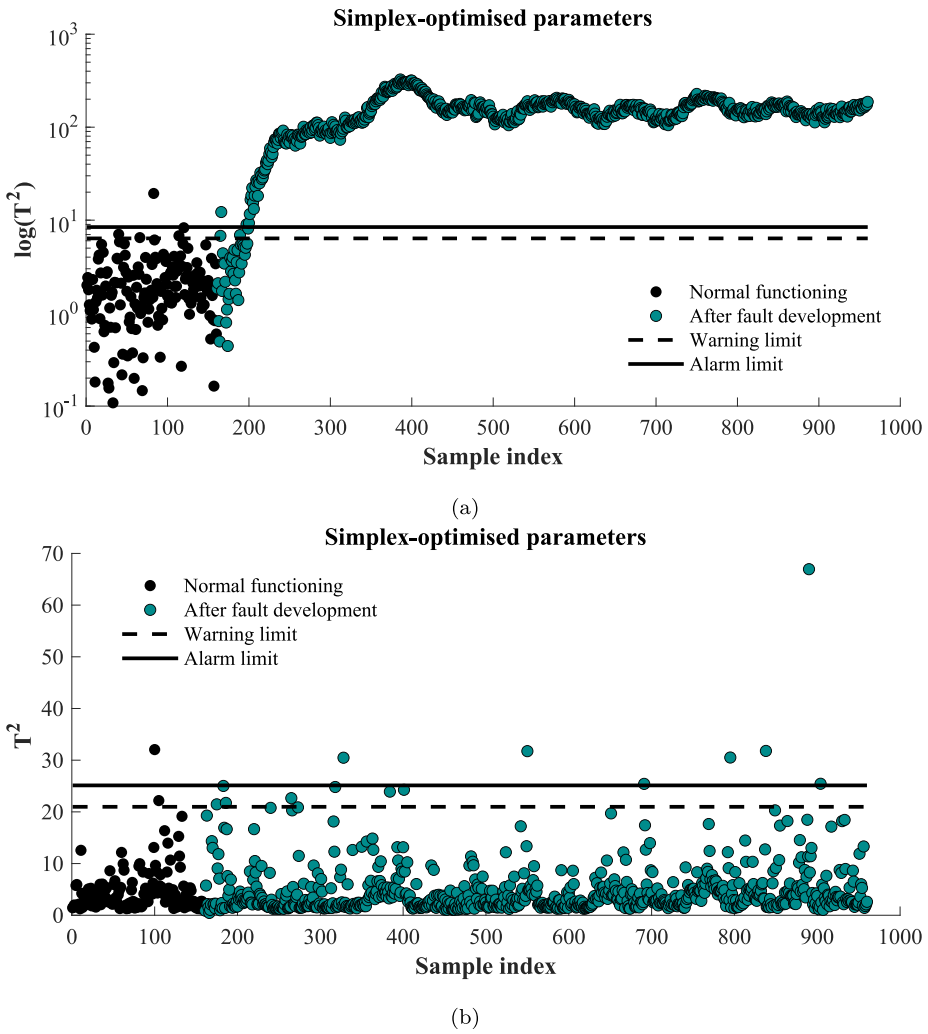


Fig. I.19. The control charts resulted from parameter optimisation with the Nelder–Mead simplex algorithm.

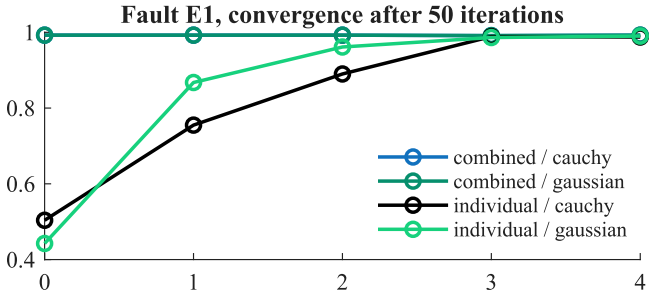


Fig. J.20. Effect of lagged variables on early convergence.

References

[1] L.-T. Zhao, T. Yang, R. Yan, H.-B. Zhao, Anomaly detection of the blast furnace smelting process using an improved multivariate statistical process control model, Process. Saf. Environ. Prot. 166 (2022) 617–627.

[2] T. Kourti, Application of latent variable methods to process control and multivariate statistical process control in industry, Internat. J. Adapt. Control Signal Process. 19 (4) (2005) 213–246.

[3] R.L. Mason, J.C. Young, Multivariate Statistical Process Control, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 928–930, http://dx.doi.org/10.1007/978-3-642-04898-2_38.

[4] S.-P. Reinikainen K. Pöllänen, J. Rantanen, P. Minkkinen, Dynamic pca-based mspc charts for nucleation prediction in batch cooling crystallization processes, Chemometr. Intell. Lab. Syst. 84 (1–2) (2006) 126–133.

[5] Z. Ge, Z. Song, Multivariate statistical process control: Process monitoring methods and applications, in: Advances in Industrial Control, Springer London, 2012.

[6] C.-F. Chien, C.-Y. Hsu, P.-N. Chen, Semiconductor fault detection and classification for yield enhancement and manufacturing intelligence, Flex. Serv. Manuf. J. 25 (2013) 367–388.

[7] G. Shi, L. Lin, Y. Liu, G. Chen, Y. Luo, Y. Wu, H. Li, Pharmaceutical application of multivariate modelling techniques: a review on the manufacturing of tablets, RSC Adv. 11 (14) (2021) 8323–8345.

[8] R.S. Kenett, S. Zacks, P. Gedeck, Multivariate Statistical Process Control, Springer International Publishing, Cham, 2023, pp. 113–140, http://dx.doi.org/10.1007/978-3-031-28482-3_4.
[9] Y.-J. Park, S.-K.S. Fan, C.-Y. Hsu, A review on fault detection and process diagnostics in industrial processes, Processes 8 (9) (2020) 1123.
[10] J. Yu, C. Zhang, S. Wang, Sparse one-dimensional convolutional neural network-based feature learning for fault detection and diagnosis in multivariable manufacturing processes, Neural Comput. Appl. 34 (6) (2022) 4343–4366.
[11] X. Kong, J. Luo, X. Feng, An overview of conventional mspc methods, Process. Monit. Fault Diagn. Based Multivariable Stat. Anal. (2024) 9–25.
[12] Z. Ge, Z. Song, An Overview of Conventional MSPC Methods, Springer London, London, 2013, pp. 5–11, http://dx.doi.org/10.1007/978-1-4471-4513-4_2.
[13] M. Grasso, B.M. Colosimo, Q. Semeraro, M. Pacella, A comparison study of distribution-free multivariate spc methods for multimode data, Qual. Reliab. Eng. Int. 31 (1) (2015) 75–96.
[14] H. Sabahno, S.T. A. Niaki, New machine-learning control charts for simultaneous monitoring of multivariate normal process parameters with detection and identification, Mathematics 11 (16) (2023) 3566, http://dx.doi.org/10.3390/math11163566.
[15] A. Yeganeh, A. Johannssen, N. Chukhrova, M. Rasouli, Monitoring multistage healthcare processes using state space models and a machine learning based framework, Artif. Intell. Med. 151 (2024) 102826.
[16] A. Yeganeh, S.C. Shongwe, A.A. Nadi, M.M. Ghuchani, Monitoring bivariate autocorrelated process using a deep learning-based control chart: A case study on the car manufacturing industry, Comput. Ind. Eng. 199 (2025) 110725.
[17] S.I. Chang, P. Ghafariasl, A review of artificial intelligence impacting statistical process monitoring and future directions, Ind. Manuf. Syst. Eng. (2025) Preprint/arXiv.
[18] T. Biegel, P. Helm, N. Jourdan, J. Metternich, Ssmspc: Self-supervised multivariate statistical in-process control in discrete manufacturing processes, J. Intell. Manuf. 35 (2023) 2671–2698, http://dx.doi.org/10.1007/s10845-023-02156-7.
[19] G.T. Jemwa, C. Aldrich, Kernel-based fault diagnosis on mineral processing plants, Miner. Eng. 19 (11) (2006) 1149–1162.
[20] M. Jia, H. Xu, X. Liu, N. Wang, The optimization of the kind and parameters of kernel function in kpca for process monitoring, Comput. Chem. Eng. 46 (2012) 94–104.
[21] H. Owhadi, G.R. Yoo, Kernel flows: From learning kernels from data into the abyss, J. Comput. Phys. 389 (2019) 22–47, http://dx.doi.org/10.1016/j.jcp.2019.03.040.
[22] Z.-S. Duma, J. Susiluoto, T. Sihvonen O. Lamminpää, S.-P. Reinikainen, H. Haario, Kf-pls: Optimizing kernel partial least-squares (k-pls) with kernel flows, Chemometr. Intell. Lab. Syst. 254 (2024) 105238, http://dx.doi.org/10.1016/j.chemolab.2024.105238.
[23] Z.-S. Duma, T. Sihvonen, J. Susiluoto, H. Haario O. Lamminpää, S.-P. Reinikainen, Kernel-based retrieval models for hyperspectral image data optimized with kernel flows, in: 2024 14th Workshop on Hyperspectral Imaging and Signal Processing: Evolution in Remote Sensing, WHISPERS, 2024.
[24] E.L. Russell, L.H. Chiang, R.D. Braatz, Fault detection in industrial processes using canonical variate analysis and dynamic principal component analysis, Chemometr. Intell. Lab. Syst. 51 (1) (2000) 81–93.
[25] S. Heo, J.H. Lee, Statistical process monitoring of the tennessee eastman process using parallel autoassociative neural networks and a large dataset, Processes 7 (7) (2019) 411.
[26] R.T. Samuel, Y. Cao, Nonlinear process fault detection and identification using kernel pca and kernel density estimation, Syst. Sci. Control. Eng. 4 (1) (2016) 165–174.
[27] T.J. Rato, M.S. Reis, Fault detection in the tennessee eastman benchmark process using dynamic principal components analysis based on decorrelated residuals (dpca-dr), Chemometr. Intell. Lab. Syst. 125 (2013) 101–108.
[28] J.J. Downs, E.F. Vogel, A plant-wide industrial process control problem, Comput. Chem. Eng. 17 (3) (1993) 245–255.
[29] L. Chiang, Fault Detection and Diagnosis in Industrial Systems, Springer Science & Business Media, 2000.
[30] R. Bro, A.K. Smilde, Principal component analysis, Anal. Methods 6 (9) (2014) 2812–2831.
[31] R. Fezai, M. Mansouri, O. Taouali, M.F. Harkat, N. Bouguila, Online reduced kernel principal component analysis for process monitoring, J. Process Control 61 (2018) 1–11.
[32] S. Maji, A.C. Berg, J. Malik, Efficient classification for additive kernel svms, IEEE Trans. Pattern Anal. Mach. Intell. 35 (1) (2012) 66–77.
[33] D.K. Duvenaud, H. Nickisch, C. Rasmussen, Additive gaussian processes, Adv. Neural Inf. Process. Syst. 24 (2011).
[34] J.I. Susiluoto O.M. Lamminpää, J.M. Hobbs, A.J. Braverman J.L. McDuffie, H. Owhadi, Forward model emulator for atmospheric radiative transfer using gaussian processes and cross validation, Atmos Meas. Tech. Discuss. 2024 (2024) 1–32.
[35] M. Kallas, G. Mourot, K. Anani, J. Ragot, D. Maquin, Fault detection and estimation using kernel principal component analysis, IFAC-PapersOnLine 50 (1) (2017) 1025–1030.
[36] J. Yu, X. Zheng, S. Wang, A deep autoencoder feature learning method for process pattern recognition, J. Process Control 79 (2019) 1–15.
[37] Y. Kanno, H. Kaneko, Deep convolutional neural network with deconvolution and a deep autoencoder for fault detection and diagnosis, ACS Omega 7 (2) (2022) 2458–2466.
[38] T. Biegel, N. Jourdan, C. Hernandez, A. Cviko, J. Metternich, Deep learning for multivariate statistical in-process control in discrete manufacturing: A case study in a sheet metal forming process, Procedia CIRP 107 (2022) 422–427.
[39] Z. Xiao, A. Kordon, S. Sen, Fault detection and diagnosis in tennessee eastman process with deep autoencoder, in: Annual conference of the phm society, Vol. 15, 2023.
[40] C. Huang, Y. Chai, Z. Zhu, B. Liu, Q. Tang, A novel distributed fault detection approach based on the variational autoencoder model, ACS Omega 7 (3) (2022) 2996–3006.
[41] A. Maged, C.F. Lui, S. Haridy, M. Xie, Variational autoencoders-lstm based fault detection of time-dependent high dimensional processes, Int. J. Prod. Res. 62 (4) (2024) 1092–1107.
[42] M. Yin, J. Tian, Y. Wang, J. Jiang, A novel distributed process monitoring framework of vae-enhanced with deep neural network, Neural Process. Lett. 56 (2) (2024) 118.
[43] J. Yu, A review for manifold learning-based statistical process control, in: 11th International Symposium on Measurement and Quality Control, Cracow and Kielce, Poland, 2013, pp. 1–3.
[44] K. Huang, H. Wen, H. Liu, C. Yang, W. Gui, A geometry constrained dictionary learning method for industrial process monitoring, Inform. Sci. 546 (2021) 265–282.
[45] J. Yu, C. Zhang, Manifold regularized stacked autoencoders-based feature learning for fault detection in industrial processes, J. Process Control 92 (2020) 119–136.